

Yanasse, H.H.; Becceneri, J.; Soma, N.Y. Bounds for a problem of sequencing patterns. *Pesquisa Operacional* **19**(2):249-277, Dezembro de 1999.

BOUNDS FOR A PROBLEM OF SEQUENCING PATTERNS

Horacio Hideki Yanasse

Laboratory for Computing and Applied Mathematics
Brazilian Space Research Institute

José Carlos Becceneri

Laboratory for Computing and Applied Mathematics
Brazilian Space Research Institute

Nei Yoshihiro Soma

Aeronautic Institute of Technology
Aerospace Technical Center

Abstract:

In some wood cutting settings, the cutting sequence of the patterns produced to minimize trim waste and satisfy customers' requirements is important. Each different part size cut is put into a stack which is removed only after the last part of that size is cut. It is desired to minimize the maximum number of stacks often because of space limitations and/or material handling problems. This problem can be viewed as a graph traversing problem and we propose a heuristic procedure that generates a solution based on an arc contraction scheme. Using this scheme, a possible way of computing a lower bound for this problem is also suggested. Limited computational results are presented to assess the quality of these bounds.

Keywords: cutting stock, pattern sequencing, pattern allocation, graph traversing

Resumo:

Em alguns ambientes de corte, a sequência de corte dos padrões produzidos, os quais minimizam as perdas de material e satisfazem as demandas dos clientes, é importante. Cada parte de tamanho diferente cortada é colocada em uma pilha que é removida somente quando a última parte daquele tamanho é cortada. Deseja-se minimizar o número máximo de pilhas abertas durante o processo de corte, muitas vezes por limitações de espaço e/ou problemas de manuseio de material. Este problema pode ser colocado como um problema de percorrimeto em grafos e propõe-se um procedimento heurístico que gera uma solução baseado em um esquema de contração de arcos. Usando este esquema, sugere-se também uma possível maneira de se computar um limitante inferior para este problema. Resultados computacionais limitados são apresentados para se avaliar a qualidades desses limitantes.

1. INTRODUCTION

In many production systems we are faced with the problem of cutting smaller items from larger objects. Cutting patterns, that is, which items and how to cut them from the larger objects, have to be determined according to some objective function (for instance, minimization of waste of material, minimization of production costs, etc) and cutting constraints (for instance, guillotine cuts, orthogonal cuts, etc) in order to obtain the desired items.

In some cases, the sequence in which the patterns are cut is important. In the glass industry, for instance, it is often desired that the glass characteristics of all the parts to be delivered to each particular client is as homogeneous as possible. Hence, due to the difficulties in maintaining the production of glass with exactly the same characteristics, it is desirable that the patterns that contain items of a given client are scheduled for cutting as close as possible to each other (Madsen (1988)). Limited storage space and material handling problems may also impose restrictions on the sequence in which the patterns are to be cut.

We consider here the minimization of open stacks problem (MOSP) (see Yanasse (1994, 1997a)) which arises, for instance, in wood cutting settings where panels (the small items) are to be cut from large boards (the object). A stack is opened for each different panel type cut and it is closed only after the last part of that type is cut. Closed stacks can be removed prior to the cutting of the next pattern. Due to limited storage space around the saw machine, the number of stacks must be kept as low as possible. We seek a cutting sequence of the patterns that minimizes the maximum number of open stacks during the production run.

The MOSP was considered in Yanasse (1997a), where a non-trivial mathematical formulation, a branch and bound algorithm and a greedy heuristic algorithm are suggested. Relations of the MOSP with the minimization of order spread problem (see Garey, Graham, Johnson & Knuth (1978); Madsen (1988)) and the minimization of the number of tool switches problem (see Tang & Denardo (1988)) are also discussed. Other production settings where MOSP appears are also indicated.

Lins (1989) proposed a heuristic method to solve the MOSP for the particular case where there are at most 2 different panel types per pattern and the graph associating the nodes to panel types and arcs to patterns is a 1-tree, i.e. a tree with an additional arc.

Yanasse (1996a) focused on the case with at most two panel types in each pattern. He presented polynomial algorithms for the case where the graph corresponding to the problem is a tree, a 1-tree or a "0-1 common vertex polygon" graph. Yuen (1991, 1995) and Yuen & Richardson (1995) propose several simple algorithms for pattern sequencing. Recently, Linhares (1998) studied the similarities of MOSP with combinatorial problems arising in VLSI. He showed that MOSP is NP-hard, by proving its equivalence with the Gate Matrix Layout Problem.

Yanasse (1997b) proved that an optimal solution to *any* MOSP can be obtained by solving a corresponding MOSP where all patterns have at most two panel types. The corresponding MOSP can be obtained in polynomial computational time, hence, solving general MOSPs is as difficult as solving "simpler" ones where all patterns have at most two panel types. Therefore, from now onwards, without loss of generality, we consider only 2MOSP instances (an instance where each pattern has at most two panel types) and, hence, the problem can be described using a graph. The graph problem equivalent to 2MOSP is an arc traversing problem: traverse all arcs of the graph such that the maximum number of simultaneously open nodes is minimized. A node is open when the first arc incident to it is traversed; a node is considered closed after the last arc incidence to it is traversed.

Since MOSP can be seen as a graph traversing problem and, in other settings, also as a task sequencing problem (see Yanasse, 1997a), we will use the terms arcs and/or tasks in place of patterns and, nodes and/or orders in place of stacks or items, without distinction in the text.

In this paper we propose a new heuristic to solve the MOSP based on an arc-contraction (see, for instance, Foulds, 1992) procedure that modifies the original graph so that a known graph topology is obtained whose optimal solution can be derived without much difficulty. A solution to the original problem is constructed on top of this surrogate graph solution. With the arc contraction procedure we also propose a new way of determining a lower bound for the MOSP. Bounds are useful in branch-and-bound schemes for solving this problem. Lower bounds are also useful to assess the quality of solutions obtained by heuristics.

We point out that the idea of using a modified graph scheme was used previously in Yanasse (1996b), with an arc-deleting procedure. The performance of the arc-deleting algorithm and Yuen's heuristics (see Yuen (1995)) are compared with the new arc-contraction method being proposed in this manuscript.

In the next section, we consider the effect of an arc contraction in the solution to MOSP. We present a result which is the basis for the proposed arc contraction algorithm and the new way of determining lower bounds for the MOSP. The heuristic and lower bound procedure are presented in section 3. In section 4 we present the computational tests performed preceded by a brief description of some heuristics and lower bounds that were used for the purpose of comparisons. Some concluding remarks are made in section 5.

2. Arc contraction operation and properties of MOSP

We assume from now on that the MOSP is given in its graphical form and, without loss of generality, that the graph is connected. Recall (see Yanasse (1997b)) that the graph corresponding to a problem instance is obtained by associating nodes to parts and there is an arc adjacent to two nodes whenever there is at least one pattern containing both parts. Consider for instance the MOSP data given in Table 1. The corresponding graph representation is presented in Figure 1, where for the purpose of illustration, we indicate the arcs associated with pattern P_1 .

{ Table 1 }

{Figure 1}

Let $G(V,A)$ be a graph where V is the set of nodes and A is the set of arcs. Let $a_1 \in A$ be incident to nodes $v_1 \in V$ and $v_2 \in V$. We define a new graph, denoted by G/a_1 , obtained by contracting arc a_1 , i.e., the resulting graph after the following operations are performed (Foulds, 1992, p. 304):

1. delete $a_1 \in A$ and merge v_1 and v_2 into a single node, say v ;
2. all arcs that are incident to nodes v_1 or v_2 are now incident to node v ;
3. remove multiple arcs leaving a single arc.

Consider again the graph given in Figure 1.

Contracting, for instance, arc (b,e) in the graph presented in Figure 1 results in the graph presented in Figure 2, where b and e were merged into node b' .

{Figure 2}

The following observation relates the MOSP corresponding to the initial graph and the one obtained by contracting one (or more) of its arcs.

Proposition 1: Let $G(V,A)$ be the corresponding graph of a MOSP $PROB_1$ and $(v1,v2)$ some arc of A . Consider $G/(v1,v2)$ where we denote by v the vertex in place of the vertices $v1 \in V$ and $v2 \in V$. Let us denote $PROB_2$ the surrogate MOSP corresponding to graph $G/(v1,v2)$. Let SEQ_1 be any sequencing of the patterns in problem $PROB_1$. Let SEQ_2 be exactly as SEQ_1 excluding the patterns corresponding to arc $(v1,v2)$ and the duplicated ones removed from graph $G(V,A)$. SEQ_2 is, hence, a feasible solution to $PROB_2$. If we follow these sequences, after each corresponding pattern sequenced is cut, the number of open stacks resulting using SEQ_1 in problem $PROB_1$ is greater or equal to the number of open stacks resulting using SEQ_2 in problem $PROB_2$.

Proof of Proposition 1: SEQ_1 contains the same arcs as in SEQ_2 except those that are incident to nodes $v1$ or $v2$. In correspondence to these arcs incident to nodes $v1$ or $v2$, we have in SEQ_2 , arcs incident to node v or no arc at all (case of arc $(v1,v2)$ and the duplicate arcs). Therefore, the number of open stacks using SEQ_1 will be at least as large as the number of open stacks using SEQ_2 since the following:

- both sequences start with no open nodes;
- every stack opened following SEQ_1 will also be opened when the corresponding pattern is sequenced following SEQ_2 , except possibly when $v1$ or $v2$ are opened. In this case, in correspondence to opening $v1$ and/or $v2$, v will be opened, remain opened or not be opened at all. Since the number of open stacks in SEQ_2 remained smaller or equal up to that point, in any of the cases, the number of open stacks will remain equal or smaller than what is observed following sequence SEQ_1 .
- every stack closed following SEQ_1 will also be closed when the corresponding pattern is sequenced following SEQ_2 , except possibly when $v1$ or $v2$ are closed. In this case, in correspondence to closing $v1$ and/or $v2$, v will be closed, will remain opened or would have been closed previously. v remains opened only when $v1$ remains opened when $v2$ is closed or vice-versa. Hence, in all cases, the number of open stacks in SEQ_2 will remain equal or smaller than following sequence SEQ_1 .

As a direct corollary of *Proposition 1*, we have:

Corollary 1: The optimal value of $PROB_2$ is a lower bound for the optimal value of $PROB_1$.

3. Lower bounds for the MOSP and an arc contraction heuristic

3.1. Lower bounds for MOSP

- **1st trivial lower bound for MOSP**

One trivial lower bound for MOSP, indicated in Yanasse (1994, 1997a), is the maximum number of different parts in a pattern. We will denote the lower bound obtained this way by LB^1 .

LB^1 = the maximum number of different parts in a pattern

Recall the instance of MOSP given in Table 1. P_1 (also, P_3 , P_4 or P_{10}) has 4 different order parts, therefore, the minimum number of stacks that will be opened in an optimal solution is 4, that is $LB^1 = 4$.

- **2nd trivial lower bound for MOSP**

Another immediate lower bound (see Yanasse 1997a), which we will denote by LB^2 , can be obtained observing that whatever the solution of this sequencing problem is, some stack must be the first to be completed. No stack is closed before the first stack is completed. Hence, a lower bound for MOSP can be obtained by comparing the number of open stacks obtained when each one of the parts in the problem is considered the first to be completely cut and choosing the smallest value. Observe that to complete some part, all the patterns that contains it must be cut.

LB^2 = the minimum of the number of open stacks obtained when each one of the parts in the problem is the first to be completely cut

Consider again the instance of MOSP given in Table 1. Some stack corresponding to part a, b, c, d, e, f, g, h, i or j must be completed first. To complete some part, all the patterns that contain it must be cut. For instance, patterns P_1 , P_4 and P_9 must be cut in order to finish part a, resulting in at least 7 open stacks, one corresponding to each different part cut. If we compute the resulting number of stacks for each one of the parts in the problem, the smallest value obtained is 2, which is obtained by completing part c. Therefore, we obtain a lower bound $LB^2 = 2$ for the optimal solution to MOSP.

In Yanasse (1997b) some solution properties of the MOSP that are useful in determining other lower bounds are presented:

Proposition 2 (Yanasse (1997b)): Suppose we have a MOSP, $PROB_1$, with N patterns to be sequenced. Suppose we have a second problem, $PROB_2$, with the same N patterns as problem $PROB_1$ plus a few more patterns. Then the optimal solution value of $PROB_2$ is greater than or equal to the optimal solution value of $PROB_1$.

Proposition 3 (Yanasse (1997b)): Let $G(V,E)$, the graph obtained from a problem $PROB$, be a connected non-empty graph where all nodes have the degree at least $n \geq 2$. Then, a lower bound on the optimal solution value to $PROB$ is $n+1$, that is, in an optimal solution to $PROB$, we must have at least $n+1$ open stacks at some time during cutting.

Observe that in *Proposition 3*, there is no particular reason to restrict the value of n to be greater than or equal to 2. This was done because lower bounds equal to 1 or 2 are trivial: any problem corresponding to a connected graph with more than one node has a lower bound of 2.

Propositions 2 and *3* are used to determine the next 3 bounds. Before we move on, we make a brief observation on optimal solutions to a MOSP when its corresponding graph is complete.

Let $G(V,E)$, the graph obtained from a problem $PROB$, be a complete graph with $n+1$ nodes. Then according to *Proposition 3*, an optimal solution value to $PROB$ is $n+1$, that is, all the $n+1$ stacks will be open at some time during cutting. Hence, in this case, any sequencing of the patterns is optimal.

- **3rd lower bound for MOSP**

Delete arcs of the original graph representation of the problem. Solve the simpler problem (simpler graph) exactly. By *Proposition 2*, its solution value is a lower bound for the original problem. We will denote a lower bound obtained this way as LB^3 .

LB^3 = optimal solution value of a “simpler” problem obtained by deleting arcs.

Consider again the instance of MOSP given in Table 1. By using *Proposition 2*, if we delete all arcs incident to node c, the resulting graph is the one presented in Figure 3.

{Figure 3}

The problem seems still complex to solve. If we delete all arcs incident to nodes b, d, e and f which are the nodes in this graph with the smallest degree, the resulting graph is presented in Figure 4.

{Figure 4}

An optimal solution to this graph problem can be obtained now without much difficulty with optimal value of 4 open stacks. This is, hence, a lower bound on the optimal solution value to the original problem, that is, $LB^3 = 4$.

- **4th lower bound for MOSP**

Use *Proposition 3* directly. Analyse the degree of the nodes of the graph representation of the problem. The lower bound obtained this way will be denoted LB^4 .

$$LB^4 = \text{smallest degree of a node in the graph} + 1.$$

In the graph of Figure 1, all nodes have degree at least 1, hence, a lower bound on the optimal solution value is 2 ($LB^4 = 2$).

Combination of Propositions 2 and 3

Due to *Proposition 2*, every subgraph of the original graph can also be considered in this analysis. So we may attempt to delete nodes (and all the arcs incidents to it) having the smallest degrees.

In the graph of Figure 1, we could attempt to delete node c and all the arcs incidents to it, since it is the node with the smallest degree. In the resulting subgraph (Figure 3) the least degree is 4, hence, a lower bound value of 5 is attained ($LB^4 = 5$). No better lower bound value is obtained by deleting more arcs in the subgraph of Figure 3.

Given a graph, the best possible lower bound LB^4 is obtained by finding a induced subgraph (can have more than one) whose minimum degree is maximized. A induced subgraph whose minimum degree is maximized can be obtained using recursively the scheme presented previously, that is, select to delete the node with the smallest degree. This is performed while all nodes of the resulting graph do not present the same degree.

- **5th lower bound for MOSP**

The basic idea used to determine an improved lower bound is:

1. Repeatedly apply the arc-contraction scheme, to the “difficult” problem until it is transformed into a graph whose optimal solution can be obtained without difficulty. Calculate LB^4 for each intermediate subgraph obtained.
2. Determine LB^5 by comparing the optimal solution value of the last reduced graph and the intermediate values of LB^4 obtained.

Consider the graph representation of the problem given in Figure 1. We are interested in stronger lower bounds, hence, we will try to contract arcs incidents to nodes with the smallest degree value vertices. Since vertex c is the one with the smallest degree value, we may contract the arc incident to it. The resulting graph is equal to the one presented in Figure 3.

No improvement of bounds (which at this moment is 5 by computing LB^4 in this graph) has been achieved at this time after contracting the first arc.

Consider now the graph in Figure 3. Since vertices b , d , e and f are the ones with the smallest degree value, we may contract, for instance, the arc that is incident to b and e . The resulting graph is presented in Figure 5.

{Figure 5}

Consider now the graph in Figure 5. No improvement of the bound is obtained. We proceed in the same way and we may contract, for instance, the arc that is incident to b and f . The resulting graph is presented in Figure 6.

{Figure 6}

Consider now the graph in Figure 6. Again, no improvement of the bound is obtained. We proceed in the same way and we may contract, for instance, the arc that is incident to d and h . The resulting graph is presented in Figure 7.

{Figure 7}

The resulting graph in Figure 7 is complete. Therefore $LB^5 = 6$, for this instance. Notice that we could obtain this bound using LB^4 in this last obtained graph since all nodes have degree at least 5.

As observed in the instance presented, the arc contraction procedure produces stronger lower bounds compared with the simple arc deletion procedure proposed previously. This happens because the contraction procedure may increase the degree of some of the remaining nodes of the graph. With the arc deletion procedure this will never happen.

We formalize next the proposed general contraction procedure for determining lower bounds for the MOSP. We assume the MOSP is given in its corresponding graph problem and recall that we are assuming, without loss of generality, that the graph is connected.

Procedure LBArcContraction

- Step 0 - Compute $LB = \max \{LB^1, LB^2, LB^3\}$, the initial lower bound for the problem and M , the number of nodes in the graph considered.
- Step 1 - Compute the degree of each one of the M nodes in the graph. Compute LB^4 . If $LB^4 > LB$, then $LB = LB^4$. If $M \leq LB$, then $LB^5 = LB$ and STOP. Else, let S be the set of nodes with the smallest degree. Choose any arc, say a , in the graph which is incident to two nodes whose sum of their degrees is the minimum and such that, at least one of the nodes belongs to set S .
- Step 2 - Contract arc a , and as the result of this arc contraction, a single supervertex is obtained in place of the two vertices incidents to a . All the arcs that were incident to these two vertices are now incident to the supervertex; in case of duplication of arcs, maintain only a single one. Set $M \leftarrow M-1$ and return to Step 1.

3.2 Arc contraction heuristic

We next present a new procedure, denoted arc-contraction heuristic, which has similarities with the arc-deleting heuristic (see Yanasse (1996b)) but uses a different reduction procedure that consists in contracting arcs instead of simply deleting them.

Our proposed heuristic determines a solution (preferably an optimal solution) for the reduced problem, say P_2 , and tries to extend this solution to the original problem, say P_1 . Observe that if this extended solution is such that it does not increase the maximum number of open stacks observed for problem P_2 and if the solution obtained for problem P_2 is optimal, we can guarantee that the solution obtained for problem P_1 is also optimal.

Initially nodes with degree 1 are identified and partial sequences that are optimal for subgraphs of the original graph that are trees are built. These optimal subsequences are constructed using the algorithm proposed in Yanasse (1997c). The algorithm for trees basically builds a solution from optimal solution of its parts. If we momentarily delete all nodes of a tree with degree 3 or more (and all adjacent arcs incident to them), the resulting forest is composed of simpler trees whose solutions are easily determined. The algorithm just combines these solutions to build an optimal solution to the original tree.

These subgraphs that are trees can then be momentarily ignored, in order to solve the remaining “complicated part” of the problem. The arc-contraction scheme is repeatedly used in this difficult problem until it is transformed into an instance whose optimal solution can be obtained without difficulty. This solution is then extended to obtain a solution for the original problem. A solution to the original graph problem is obtained inserting conveniently in the (optimal) sequence obtained the contracted arcs that were deleted and the accompanying arcs (duplicated arcs) that were omitted after the contraction operations.

The contraction operations will lead to complete graphs having eventually some tree subgraphs attached to its nodes (see Figure 8).

{Figure 8}

We provide next an exact algorithm for MOSP with this particular topology.

Exact algorithm for complete graphs having eventually some tree subgraphs attached to its nodes

We consider initially simpler cases:

Case I) Complete graph

If the graph is complete, any sequencing of the patterns is an optimal solution since to finish any node, all other nodes will be opened.

Case II) A complete subgraph with subtrees attached to its nodes, **exactly one** to each one of the nodes

If there are subtrees attached to some of the nodes then not all sequencing of the patterns lead to an optimal sequence of the problem.

Assume we have an n -node complete subgraph $G(N,A)$ with Subtree₁ attached to node 1, Subtree₂ attached to node 2, ..., Subtree _{n} attached to node n , respectively, where 1, 2, ..., n are the nodes of subgraph $G(N,A)$. Let us assume $PW_1, PW_2, \dots, PW_n$ be the optimal solutions of the MOSPs corresponding to subtrees Subtree₁, Subtree₂, ..., Subtree _{n} , respectively. Let us assume $pw_1, pw_2, \dots, pw_n$, respectively, are their associated optimal solution values.

Let us denote $E\text{Subtree}_i$, the subtree composed of Subtree_i together with node i , and arc (v_i, i) , where v_i is the node in Subtree_i that is adjacent to node i . Let FPW_i be an optimal sequence of the arcs in $E\text{Subgraph}_i$ with the restriction that once vertex i is opened, it remains opened from that point onwards. Let fpw_i be its corresponding optimal solution value. FPW_i can be obtained modifying slightly the exact algorithm for trees of Yanasse (1997c). Observe that, for all i , $\text{pw}_i+1 \geq \text{fpw}_i$ since $\text{PW}_i[(v_i, i)]$ is a feasible sequence for subgraph $E\text{Subtree}_i$ which produces at most pw_i+1 open orders, where $\text{PW}_i[(v_i, i)]$ denotes the sequence PW_i with arc (v_i, i) inserted just after the first time node v_i is opened.

Without loss of generality, let $(\text{fpw}_1, \text{pw}_1) \geq (\text{fpw}_2, \text{pw}_2) \geq \dots \geq (\text{fpw}_n, \text{pw}_n)$, where by $(\text{fpw}_i, \text{pw}_i) \geq (\text{fpw}_j, \text{pw}_j)$ we mean $\text{fpw}_i \geq \text{fpw}_j$ **and** $\text{pw}_i \geq \text{pw}_j$.

With all the previous conditions, an optimal solution can be obtained using the following algorithm:

Procedure CompleteGraph1

The order in which the nodes in the complete subgraph $G(N, A)$ are opened is defined according to the values of their associated pair $(\text{fpw}_i, \text{pw}_i)$: node with associated largest value (node 1) is first, third largest (node 3) is second, fifth largest (node 5) is third, and so on. After all the odd nodes in the complete graph are opened, then sequence all arcs necessary to close all these odd nodes. As a result, all even nodes of the complete graph will be opened. The order in which the even nodes in the complete subgraph $G(N, A)$ are closed is defined according to the values of their associated pair $(\text{fpw}_i, \text{pw}_i)$: even node with associated largest value (node 2) is last, second largest (node 4) is the one before the last, and so on. The established order in which the odd nodes in the complete graph are opened defines the order in which the FPW_i , $i = 1, 3, 5, \dots$ are sequenced in the final optimal sequence. The established order in which the nodes in the complete graph are closed defines the order in which the revFPW_i , $i = 2, 4, 6, \dots$ are sequenced in the final optimal sequence, where revFPW_i denotes FPW_i in reverse order. The resulting optimal sequence constructed is given by: $\text{FPW}_1 \text{FPW}_3 \text{FPW}_5 \text{FPW}_7 \dots (1,3) (1,5) (3,5) \dots (6,4) (6,2) (4,2) \dots \text{revFPW}_6 \text{revFPW}_4 \text{revFPW}_2$, where all the arcs of the complete subgraph $G(N, A)$ are introduced in the sequence just after the last FPW_i subsequence, with i an odd index, is sequenced.

Proposition 4: Procedure CompleteGraph1 provides an optimal solution for the special case of MOSP where we have an n -node complete subgraph $G(N, A)$ with n subtrees attached to its nodes, **exactly one** to each one of the nodes, and with the characteristics and conditions stated previously.

Proof of Proposition 4: The solution value resulting from the sequence $\text{FPW}_1 \text{FPW}_3 \text{FPW}_5 \text{FPW}_7 \dots (1,3) (1,5) (3,5) \dots (6,4) (6,2) (4,2) \dots \text{revFPW}_6 \text{revFPW}_4 \text{revFPW}_2$ is given by:

$$\max \{ \text{fpw}_1, \text{fpw}_3+1, \text{fpw}_5+2, \text{fpw}_7+3, \dots, n, \dots, \text{fpw}_6+2, \text{fpw}_4+1, \text{fpw}_2 \}.$$

In any sequencing of the patterns, the nodes of the complete subgraph (nodes belonging to N) once opened remain opened until all of the nodes of N are opened, and, in addition, a node can be closed only if the arc linking it to its corresponding subtree is also sequenced. Therefore, we must sequence these arcs in order to close the nodes of the complete graph. If we start the sequencing with some arc in the complete subgraph (some arc belonging to A), at least one node belonging to N will be open when we move on and sequence the arcs of any one of the subtrees. If, on the other hand, we start with an arc

of one of the subtrees, no nodes of N will be yet opened. Due to the symmetry of the solution to this problem, the best we can do is to have no open nodes when sequencing the arcs of the first two of the subtrees, if one of the subtrees is traversed first and the other is traversed last. Also, it is inevitable that we will have at least one open node when sequencing the arcs of the next two subtrees, since at least one node of the initial (last) subtree or a node of N will remain opened. In a similar way, we will have at least 2 open nodes when sequencing the arcs of the following next two subtrees, at least 3 open nodes when sequencing the arcs of the following next two subtrees and so on. Therefore, it is inevitable that the number of open stacks that are going to be formed will be at least $\max\{fpw(1), fpw(2)+1, fpw(3)+2, fpw(4)+3, \dots, n, \dots, fpw(n-2)+2, fpw(n-1)+1, fpw(n)\}$, where $fpw(i)$ is the i -th node chosen to be opened in N for $i \leq n/2$ or closed for $i > n/2$. The order suggested in the procedure minimizes this value, that is, $\max\{fpw_1, fpw_3+1, fpw_5+2, fpw_7+3, \dots, n, \dots, fpw_6+2, fpw_4+1, fpw_2\} = \min\{\max\{fpw(1), fpw(2)+1, fpw(3)+2, fpw(4)+3, \dots, n, \dots, fpw(n-2)+2, fpw(n-1)+1, fpw(n)\}\}$ for all possible permutations, therefore, the solution obtained is optimal.

Observe that when there are no subtrees “attached” to a node in the complete graph they can be considered as having associated an “empty” subtree with $fpw_i = pw_i = 0$.

We next extend this same reasoning for the general case.

Case III) A complete subgraph with subtrees attached to its nodes

Consider an n -node complete subgraph $G(N,A)$ with subtrees attached to its nodes. Let us assume, without loss of generality, that the nodes in $G(N,A)$ are $1, 2, \dots, n$. For each Subtree $_i$, let PW_i be the optimal solution of the MOSP corresponding to subtree Subtree $_i$ and pw_i be its associated optimal solution value. Let us denote ESubtree $_i$, the subtree composed of Subtree $_i$ together with node r_i , the node in the complete subgraph where Subtree $_i$ is attached by some linking arc, say (v_i, r_i) . v_i therefore is the node in Subtree $_i$ that is adjacent to node r_i . Let FPW_i be an optimal sequence of the tasks in ESubgraph $_i$ with the restriction that once vertex r_i is opened, it remains opened from that point onwards. Let fpw_i be its corresponding optimal solution value. As previously observed, for all i , $pw_i+1 \geq fpw_i$. Let $S_1, S_2, \dots, S_n$ be the sets of the optimal solution values of the MOSPs corresponding to subtrees attached to nodes $1, 2, \dots, n$, respectively. Observe that some of these sets might be empty.

Let the nodes $1, 2, \dots, m$ having at least one subtree attached to them be ordered in a list L in non-decreasing order of their associated largest (fpw_i, pw_i) values. In other words, compare the largest (fpw_i, pw_i) values associated to each one of the nodes. The node having a subtree attached to it with the largest (fpw_i, pw_i) values is the first in list L , the node with the second largest values of (fpw_i, pw_i) is the second in the list and so on. In case of ties, i.e., the largest (fpw_i, pw_i) values associated to the nodes are the same, the associated second, third, fourth and so on, largest (fpw_i, pw_i) values of the subtrees attached to the nodes are compared.

Recall that $PW_{(i)}[(v_{(i)}, i)]$ is $PW_{(i)}$ with the arc $(v_{(i)}, i)$ inserted just after the first time node $v_{(i)}$, the node in Subtree $_{(i)}$ which is adjacent to node i , is opened.

An optimal solution can be obtained according to the following:

Procedure CompleteGraph

Step 0 Let $r \leftarrow 1$, $flag \leftarrow 0$. Build ordered list L as previously described.

Step 1 While the ordered list L of nodes is not empty, take out the next element in the list, say node i . Get also the next two elements in list L , say nodes j and k . If there are less than two elements in list L , get just the nodes left in the list. Let $(fpw(1), pw(1))$, $(fpw(2), pw(2))$ and $(fpw(3), pw(3))$ be the largest, second largest and third largest values in sets $S_i \cup S_j \cup S_k$. If $((fpw(1) = fpw(2) > fpw(3))$ and $((fpw(1), pw(1))$ and $(fpw(2), pw(2))$ are in S_i)) then

If (flag = 0) then

sequence $FPW_{(1)}$ in the r -th order followed by **all** $PW_{(i)}[(v_{(i)}, i)]$'s corresponding to the subtrees which are attached to the complete subgraph $G(N, A)$ through node i , **except** for $Subtree_{(2)}$. Sequence in the r -th order counting from the end, $revFPW_{(2)}$. Set $r \leftarrow r+1$, return to begin of while.

If (flag = 1) then

sequence in the r -th order counting from the end, $PW_{(i)}[(v_{(i)}, i)]$'s corresponding to **all** the subtrees which are attached to the complete subgraph $G(N, A)$ through node i , **except** for $Subtree_{(1)}$ **and** $Subtree_{(2)}$. Sequence $revFPW_{(1)}$. Set $r \leftarrow r+1$. Sequence $FPW_{(2)}$ in the r -th order, return to begin of while.

else

If (flag = 0) then

sequence $FPW_{(1)}$ in the r -th order followed by **all** $PW_{(i)}[(v_{(i)}, i)]$'s corresponding to the subtrees which are attached to the complete subgraph $G(N, A)$ through node i . Set flag $\leftarrow 1$, return to begin of while.

If (flag = 1) then

sequence in the r -th order counting from the end (that is, last if $r = 1$, one before last if $r = 2$, and so on), $PW_{(i)}[(v_{(i)}, i)]$'s corresponding to **all** the subtrees which are attached to the complete subgraph $G(N, A)$ through node i , **except** for $Subtree_{(1)}$. Sequence last $revFPW_{(1)}$. Set $r \leftarrow r+1$ and flag $\leftarrow 0$, return to begin of while.

Step 2 If (flag = 1) set $k \leftarrow r$ else set $k \leftarrow r-1$. Following the subsequence generated insert all the arcs in A **just after** the k -th $FPW_{(h)}$ $PW_{(i)}[(v_{(i)}, i)]$'s block is sequenced (at this point we will have k nodes of N opened).

In this procedure, “flag” is used to mark whether the next node is to be opened (flag = 0) or closed (flag = 1); “ r ” is used to count the order of opening (flag = 0) or closing (counted from the end, when flag = 1).

Proposition 5: Procedure CompleteGraph provides an optimal solution for the special case of MOSP where we have an n -node complete subgraph $G(N, A)$ with subtrees attached to its nodes.

Proof of Proposition 5: We follow a similar reasoning as in the proof of *Proposition 4*. Any node of the complete subgraph once opened remains opened until all of them are open, and, in addition, a node can be closed only if all the arcs linking it to its corresponding subtrees are sequenced, hence, we must sequence all these arcs in order to close the nodes of the complete graph. As we argued previously the minimum number of open nodes when sequencing the arcs of the first two of the subtrees having the largest and second largest values is no nodes, then at least one open node when sequencing the arcs of the next two subtrees with the third and fourth largest values, and so on. At least one additional node will be opened when we sequence the arcs of the next two subtrees of the list. Considering that there might be more than one subtree attached to the same node, it is important to observe whether there are subtrees with the largest value attached to the same

node. Special care has to be taken in this case in order to guarantee that the number of open stacks remains as low as possible. This is what is taken into consideration in the procedure. If the second largest value attached to the same node is strictly smaller than the largest value, then, after we sequence first the arcs of the subtree with the largest value, we can sequence the arcs of all remaining subtrees attached to that node without exceeding the the largest value of first subtree sequenced. Also, the same thing can be done if the second and third largest values are equal to the largest value, since, in this case, it is inevitable that the number of open stacks will be, at least, the largest value plus one. The rest of the procedure follows exactly *Procedure CompleteGraph1*, hence, the argument used to prove the correctness of that procedure is valid here.

We illustrate Procedure CompleteGraph with an example. Consider the instance of MOSP presented in Figure 8. Let's assume $(fpw_1, pw_1) = (3, 2)$, $(fpw_2, pw_2) = (2, 1)$, $(fpw_3, pw_3) = (3, 2)$, $(fpw_4, pw_4) = (5, 4)$, $(fpw_5, pw_5) = (5, 4)$, $(fpw_6, pw_6) = (5, 4)$.

$S_1, S_2, \dots, S_5$ are the sets of the optimal solution values of the MOSPs corresponding to subtrees attached to nodes 1, 2, ..., 5, respectively. Therefore, S_1 is empty, S_2 has (fpw_4, pw_4) , S_3 has (fpw_5, pw_5) and (fpw_6, pw_6) , S_4 is empty, S_5 has (fpw_1, pw_1) , (fpw_2, pw_2) and (fpw_3, pw_3) . Applying Procedure CompleteGraph we obtain:

- Step 0** The ordered list L becomes $\{3, 2, 5\}$, since the largest values (fpw_5, pw_5) of node 3 is equal to the largest values (fpw_4, pw_4) of node 2, but the second largest values (fpw_6, pw_6) of node 3 is greater than the second largest values $(0, 0)$ (no second subtree attached) of node 2. Nodes 4 and 1 are not in list L since they have no subtrees attached to them. r is set to 1 and flag is set to 0;
- Step 1** The next node in list L is 3. The next two nodes in list L are 2 and 5. The largest, second largest, third largest values in $S_3 \cup S_2 \cup S_5$ are (fpw_5, pw_5) , (fpw_4, pw_4) and (fpw_6, pw_6) . Since $fpw_5 = fpw_6 = fpw_4 > 0$ and $flag = 0$, FPW_5 is sequenced first followed by $PW_6[(v_6, 3)]$. flag is updated to 1;
- Step 1** The next node in the list L is 2. The next single node in list L is 5. The largest, second largest, third largest values in $S_2 \cup S_5$ are (fpw_4, pw_4) , (fpw_3, pw_3) and (fpw_1, pw_1) , respectively. Since $fpw_4 > fpw_3$ and $flag = 1$, $revFPW_4$ is sequenced last. r is updated to 2; flag is updated to 0;
- Step 1** The next node in the list L is 5. List L is empty. The largest, second largest, third largest values in S_5 are (fpw_1, pw_1) , (fpw_3, pw_3) and (fpw_2, pw_2) , respectively. Since $fpw_1 = fpw_3 > fpw_2$, (fpw_1, pw_1) , (fpw_3, pw_3) are in S_5 and $flag = 0$, FPW_1 is sequenced in the 2nd order followed by $PW_2[(v_2, 5)]$. $revFPW_3$ is sequenced one before last. r is updated to 3.
- Step 2** flag is equal to 0, hence k is set to 2. The subsequence we have so far is $FPW_5 PW_6[(v_6, 3)] FPW_1 PW_2[(v_2, 5)] revFPW_3 revFPW_4$. We insert all arcs of A just after the second ($k = 2$) $FPW_{(h)} PW_{(i)}[(v_{(i)}, i)]$'s block is sequenced, which in this case is $FPW_1 PW_2[(v_2, 5)]$. We obtain: $FPW_5 PW_6[(v_6, 3)] FPW_1 PW_2[(v_2, 5)] (1, 2) (1, 3) (1, 4) (1, 5) (2, 3) (2, 4) (2, 5) (3, 4) (3, 5) (4, 5) revFPW_3 revFPW_4$. The resulting maximum number of open stacks is 5.

We are now ready to present the arc-contraction heuristic. Without loss of generality, we assume that the graph being considered is connected.

Arc-contraction heuristic

Step 0 Starting with the nodes with degree smaller or equal to 1, calculate all FPW_i , PW_i with corresponding values fpw_i , pw_i of the components of the graph that are trees. If there are no more unlabelled nodes **STOP**. Compute the newscore of all unlabelled nodes.

Comment: The PW_i 's (FPW_i 's) can be obtained using (an adaptation of) the algorithm for trees preented in Yanasse (1997c). At the end of **Step 0**, the procedure stops with an optimal solution if the original graph was a tree. The “newscore” attached to each node k is a two-tuple where the first element is the largest fpw -degree among all subtrees linked to node k and the second element is the number of additional subtrees linked to this node that achieves this maximum value, that is:

$\text{newscore}(k) = (\text{largest } fpw\text{-degree}, \# \text{ of additional subtrees with largest } fpw\text{-degree}).$

Step 1 Set $m \leftarrow 1$, $a^m = -1$ and denote Subgraph^m , the current subgraph under consideration.

While Subgraph^m is not complete do:

Let S^m be the set of nodes in Subgraph^m with the smallest degree. Choose any arc a^m in the graph which is incident to two nodes whose sum of their degrees is the minimum and such that at least one of the nodes belongs to S^m . If there is a tie, choose an arc incident to nodes having the smallest newscore associated with them. If the tie persists break it at random. Contract arc a^m , and as the result of this arc contraction, a single supervertex v^m is obtained in place of the two vertices incidents to a^m . Update the newscore of the supervertex v^m . Set $m \leftarrow m + 1$ and denote the graph obtained as Subgraph^m .

Comment: At the end of **Step 1**, the subgraph obtained is complete with several subtrees attached to its nodes. We perform next operations similar to Procedure CompleteGraph to determine an order to open (and close) the nodes of this modified graph. Subsequences $VSEQ^1$ and $VSEQ^2$ are the results of this ordering ($VSEQ^1$ for opening nodes and $VSEQ^2$ for closing nodes). They are used as frames to compose a solution to the problem.

Step 2 Build subsequences $VSEQ^1$ and $VSEQ^2$ as follows:

- Let $r \leftarrow 1$, $\text{flag} \leftarrow 0$. Build ordered list L according to the non-increasing values of the newscors.
- While the ordered list L of nodes is not empty, take out the next element in the list, say node i . Get also the next two elements in list L , say nodes j and k . If there are less than two elements in list L get only the ones that are left. Let $(fpw(1), pw(1))$, $(fpw(2), pw(2))$ and $(fpw(3), pw(3))$ be the largest, second largest and third largest values in sets S_i , S_j and S_k of the optimal solution values of the subtrees attached to nodes i , j and k , respectively. If $(fpw(1) > fpw(2))$ then

If ($\text{flag} = 0$) then

sequence $FPW(1)$ in $VSEQ^1$ in the r -th order followed by **all** $PW_{(s)}[(v_{(s)}, s)]$'s corresponding to the subtrees which are attached to Subgraph^1 through node s (node s is the node where the $\text{Subtree}(1)$ corresponding to $FPW(1)$ is attached to the initial subgraph, before any contraction of nodes; it may be the node i itself or some other node that was merged into node i). Set $\text{flag} \leftarrow 1$, return to begin of while.

If ($\text{flag} = 1$) then

sequence $FPW(1)$ in $VSEQ^2$ in the r -th order followed by **all** $PW_{(s)}[(v_{(s)}, s)]$'s corresponding to the subtrees which are attached to Subgraph^1 through node s (node

s is the node where the $Subtree(1)$ corresponding to $FPW(1)$ is attached to the initial subgraph, before any contraction of nodes; it may be the node i itself or some other node that was merged into node i). Set $r \leftarrow r+1$ and $flag \leftarrow 0$, return to begin of while.

If $(fpw(1) = fpw(2) > fpw(3))$ then

If $(flag = 0)$ then

sequence $FPW(1)$ in $VSEQ^1$ in the r -th order followed by **all** $PW(s)[(v(s),s)]$'s corresponding to the subtrees which are attached to $Subgraph^1$ through node s (same as described previously), **except** for $Subtree(2)$.

If $(fpw(2), pw(2))$ is in set S_i , then

sequence $FPW(2)$ in $VSEQ^2$ in the r -th order followed by **all** $PW(s)[(v(s),s)]$'s not yet sequenced corresponding to the subtrees which are attached to $Subgraph^1$ through node s (node s is the node where the $Subtree(2)$ corresponding to $FPW(2)$ is attached to the initial subgraph, before any contraction of nodes; it may be the node i itself or some other node). Set $r \leftarrow r+1$, return to begin of while.

else

set $flag \leftarrow 1$, return to begin of while.

If $(flag = 1)$ then

sequence $FPW(1)$ in $VSEQ^2$ in the r -th order followed by **all** $PW(s)[(v(s),s)]$'s corresponding to the subtrees which are attached to $Subgraph^1$ through node s (same as described previously), **except** for $Subtree(2)$. Set $r \leftarrow r+1$.

If $(fpw(2), pw(2))$ is in set S_i , then

sequence $FPW(2)$ in $VSEQ^1$ in the r -th order followed by **all** $PW(s)[(v(s),s)]$'s not yet sequenced corresponding to the subtrees which are attached to $Subgraph^1$ through node s (node s is the node where the $Subtree(2)$ corresponding to $FPW(2)$ is attached to the initial subgraph, before any contraction of nodes; it may be the node i itself or some other node that was merged into node i). Return to begin of while.

else

set $flag \leftarrow 0$, return to begin of while.

If $((fpw(1) = fpw(2) = fpw(3) = \dots = fpw(g)))$ then

If $(flag = 0)$ then

sequence $FPW(1)$ in $VSEQ^1$ in the r -th order followed by **all** $PW(s)[(v(s),s)]$'s corresponding to the subtrees which are attached to $Subgraph^1$ through node s (same as described previously). Set $flag \leftarrow 1$. If the subsequence $PW(h)[(v(h),h)]$ of any of the subtrees $Subtree(h)$, $h = 2, 3, \dots, g$, with optimal values in set S_i was not sequenced then update the newscore of node i removing all the subtrees attached to it that were already sequenced and put node i again on the top of list L . Return to begin of while.

If $(flag = 1)$ then

sequence $FPW(1)$ in $VSEQ^2$ in the r -th order followed by **all** $PW(s)[(v(s),s)]$'s corresponding to the subtrees which are attached to $Subgraph^1$ through node s (same as described previously). Set $r \leftarrow r+1$ and $flag \leftarrow 0$. If the subsequence

$PW_{(h)}[(v_{(h)}, h)]$ of any of the subtrees $Subtree_{(h)}$, $h = 2, 3, \dots, g$, with optimal values in set S_i was not sequenced then update the newscore of node i removing all the subtrees attached to it that were already sequenced and put node i again on the top of list L . Return to begin of while.

Step 3 If $VSEQ^1$ is empty and m equals to 1, then sequence all the arcs not yet sequenced in any order and **STOP**. If $VSEQ^1$ is empty and m is greater than 1, then make $VSEQ^1$ equal to a^1 , $VSEQ^2$ remains empty and go to **Step 4**. If (flag = 1) set $k \leftarrow r$ else $k \leftarrow r-1$.

Step 4 Let T^1 and T^2 be the sets of nodes opened using subsequences $VSEQ^2$ and $VSEQ^1$, respectively. Let $VSEQO^1 = VSEQ^1$ and $VSEQO^2 = VSEQ^2$. Following the subsequences $VSEQ^1$ and $VSEQ^2$ obtained so far, insert just after each one of the k first open nodes, all possible arcs that are incident solely to open nodes (both ends).

Choose among options A and B the one that leads to the smallest value of open nodes.

Option A

1. Following the subsequence $VSEQ^1$ obtained so far determine which open node, that does not belong to T^1 , that leads to only one new open node in order to complete it. If there is no such node, go to (3). Among these nodes, select node i , the one that to finish it will open a node j that has attached to it a subtree $Subtree_{(f)}$, if any (not yet sequenced in $VSEQ^1$ and $VSEQ^2$), with the largest value of fpw-degree. Insert in $VSEQ^1$, just after the first point identified in this sequence where node i can be finished with only one new open node j , $FPW_{(f)}$ followed by (i, j) .
2. Following the subsequence $VSEQ^1$ obtained so far insert all possible arcs of $Subgraph^1$ not yet sequenced in $VSEQ^1$ that are incident solely to open nodes (both ends). Following the subsequence $VSEQ^1$ obtained so far insert **all** $PW_{(s)}[(v_{(s)}, j)]$'s, if any, corresponding to the subtrees attached to node j , just after the last arc incident to node j is sequenced. Return to (1).
3. While there are still arcs not yet sequenced do:
 - Following the subsequence $VSEQ^1$ obtained so far determine which open node, say i , that does not belong to T^1 , that requires the least number of new open nodes in order to complete it. Among the new nodes that are opened to finish i , select node j (if any) that has attached to it a subtree $Subtree_{(f)}$ (not yet sequenced in $VSEQ^1$ and $VSEQ^2$) with the largest value of fpw-degree. Insert after $VSEQ^1$, $FPW_{(f)}$ followed by (i, j) , followed by **all** $PW_{(s)}[(v_{(s)}, j)]$'s, if any, corresponding to the subtrees attached to node j .
 - Following the subsequence $VSEQ^1$ obtained so far insert all possible arcs not yet sequenced in $VSEQ^1$ that are incident solely to open nodes (both ends). Return to the beginning of while.

Amend together $VSEQ^1$ followed by $revVSEQO^2$ and **STOP**.

Option B

Option B follows similar steps as in Option A using $VSEQ^2$ instead of $VSEQ^1$.

In Options A (and B), in parts 1 and 3, we choose the node having attached to it the subtree with the largest value of fpw-degree because we want to maintain the number of open nodes as low as possible. At that stage of the sequencing, the number of open nodes is likely to increase, therefore, it is

better to sequence the arcs of the subtree with the largest fpw-degree at that point (later on, the number of open nodes is likely to be larger).

We illustrate the use of the *Arc-contraction* algorithm using the instance of the problem given in Figure 9.

{Figure 9}

Following the steps of the algorithm, we obtain:

Step 0 After applying the algorithm for trees, we obtain the pair FPW_i and PW_i and corresponding values fpw_i and pw_i , respectively for $i = 1, 2, 3, 4, 5, 6$. Let us assume that $(fpw_1, pw_1) = (3, 2)$, $(fpw_2, pw_2) = (2, 1)$, $(fpw_3, pw_3) = (3, 2)$, $(fpw_4, pw_4) = (5, 4)$, $(fpw_5, pw_5) = (5, 4)$, $(fpw_6, pw_6) = (5, 4)$.

Step 1 $m \leftarrow 1$, $a^1 = -1$ and $Subgraph^1$, is the subgraph presented in Figure 10.

{Figure 10}

We now proceed with the While loop:

$Subgraph^1$ is not complete. $S^1 = \{7, 8, 11, 13\}$. Choose a^1 equal, for example, to arc (6,7). Contract arc a^1 , and as the result of this arc contraction, a single supervortex v^1 is obtained (vertex 1, in Figure 11) in place of the two vertices incidents to a^1 (vertices 6 and 7). All the arcs that were incident to these two vertices are now incident to supervortex 1. newscore of the supervortex 1 equal to (0,0). Set $m \leftarrow 2$ and denote the graph obtained as $Subgraph^2$ (see Figure 11).

{Figure 11}

$Subgraph^2$ is not complete. $S^2 = \{8, 11, 13\}$. Choose a^2 equal, for example, to arc (8,9). Contract arc a^2 , and as the result of this arc contraction, a single supervortex v^2 is obtained (vertex 2, in Figure 12) in place of the two vertices incidents to a^2 (vertices 8 and 9). All the arcs that were incident to these two vertices are now incident to supervortex 2; all the subtrees attached to these two vertices are now attached to supervortex 2, hence, newscore of the supervortex 2 is equal to (5,0). Set $m \leftarrow 3$ and denote the graph obtained as $Subgraph^3$ (see Figure 12).

{Figure 12}

We proceed with the while loop, contracting the arcs (10,11) and (12,13), obtaining supervertices v^3 and v^4 (vertices 3 and 4, respectively, in Figure 13). newscore of the supervortex 3 and 4 are equal to (5,1) and (0,0), respectively. We contract next the arc (14,15), obtaining supervortex v^5 (vertex 17 in Figure 13). newscore of the supervortex 17 is equal to (3,1). m is now equal to 6 and the graph obtained is $Subgraph^6$ (see Figure 13). We proceed with the while loop, contracting the arc (17,16), obtaining supervortex v^6 (which we will denote vertex 5). Set $m \leftarrow 7$ and denote the graph obtained as $Subgraph^7$ (the subgraph schematized in Figure 8; all subtrees attached to its nodes are represented).

Subgraph⁷ is complete, we go to **Step 2**.

{Figure 13}

Step 2 We will build subsequences $VSEQ^1$ and $VSEQ^2$.

- Let $r \leftarrow 1$, $flag \leftarrow 0$. Ordered list L is $\{3,2,5\}$.
- While loop:

Take out the next element in the list, node 3. The next two elements in the list are nodes 2 and 5. $(fpw(1), pw(1)) = (fpw_5, pw_5) = (5,4)$, $(fpw(2), pw(2)) = (fpw_6, pw_6) = (5,4)$ and $(fpw(3), pw(3)) = (fpw_4, pw_4) = (5,4)$ are the largest, second largest and third largest values in sets S_3 , S_2 and S_5 .

Since $(fpw(1) = fpw(2) = fpw(3))$ and $(flag = 0)$ then

$VSEQ^1 = FPW(1) = FPW_5$.

$flag$ is set to 1. Since $PW_6[(v_6,11)]$ was not yet sequenced, we update the newscore of node 3 and put it again on the top of list L . Return to the begin of while.

Take out the next element in the list, node 3. The next two elements in the list are nodes 2 and 5. $(fpw(1), pw(1)) = (fpw_6, pw_6) = (5,4)$, $(fpw(2), pw(2)) = (fpw_4, pw_4) = (5,4)$ and $(fpw(3), pw(3)) = (fpw_3, pw_3) = (3,2)$ are the largest, second largest and third largest values in sets S_3 , S_2 and S_5 .

Since $(fpw(1) = fpw(2) > fpw(3))$ and $(flag = 1)$ then

$VSEQ^2 = FPW(1) = FPW_6$.

r is set to 2, since $Subtree_4$ is not in S_3 then $flag$ is set to 0 and return to begin of while.

The ordered list L of nodes is not empty. Take out the next element in the list, node 2. The next single element in the list is node 5. $(fpw(1), pw(1)) = (fpw_4, pw_4) = (5,4)$, $(fpw(2), pw(2)) = (fpw_3, pw_3) = (3,2)$ and $(fpw(3), pw(3)) = (fpw_1, pw_1) = (3,2)$ are the largest, second largest and third largest values in sets S_2 and S_5 .

Since $(fpw(1) > fpw(2))$ and $(flag = 0)$ then

$VSEQ^1 = FPW_5 FPW(1) = FPW_5 FPW_4$.

$flag$ is set to 1, return to begin of while.

The ordered list L of nodes is not empty. Take out the next element in the list, node 5. No more elements in the list. $(fpw(1), pw(1)) = (fpw_1, pw_1) = (3,2)$, $(fpw(2), pw(2)) = (fpw_3, pw_3) = (3,2)$ and $(fpw(3), pw(3)) = (fpw_2, pw_2) = (2,1)$ are the largest, second largest and third largest values in set S_5 .

Since $(fpw(1) = fpw(2) > fpw(3))$ and $(flag = 1)$ then

$VSEQ^2 = FPW_6 FPW(1) = FPW_6 FPW_1$;

r is set to 3.

$VSEQ^1 = FPW_5 FPW_4 FPW(2) PW_2[(v_2,14)] = FPW_5 FPW_4 FPW_3 PW_2[(v_2,14)]$.

$flag$ is set to 1, return to begin of while.

The ordered list L of nodes is empty.

Step 3 $VSEQ^1$ is not empty, $flag$ equals 1, therefore k is set to r which is equal to 3.

Step 4 $T^1 = \{11, 15\}$. $T^2 = \{8, 10, 14\}$. Initial $VSEQ^1 = VSEQO^1 = FPW_5 FPW_4 FPW_3 PW_2[(v_2, 14)]$. Initial $VSEQ^2 = VSEQO^2 = FPW_6 FPW_1$. Following subsequence $VSEQ^1$ obtained so far, we insert just after each one the k first open nodes, all possible arcs that are incident solely to open nodes:

$$VSEQ^1 = FPW_5 FPW_4 (8, 10) FPW_3 PW_2[(v_2, 14)].$$

Following subsequence $VSEQ^2$ obtained so far, we insert just after each one the r first open nodes, all possible arcs that are incident solely to open nodes. In this case, none. Therefore $VSEQ^2$ remains equal to $FPW_6 FPW_1$.

Option A

1. Following the current $VSEQ^1$ subsequence, opening node 8 leads to one additional open node in order to complete it. We insert in $VSEQ^1$, in the proper place determined to finish node 8, arc (8,9), obtaining $VSEQ^1 = FPW_5 FPW_4 (8, 10) (8, 9) FPW_3 PW_2[(v_2, 14)]$.
2. Following the subsequence $VSEQ^1$ obtained so far, no more arcs can be inserted that are incident solely to open nodes. No arcs in this case. Return to (1).
1. Following the current $VSEQ^1$ subsequence, opening nodes 9, 10 and 14 lead to three additional open nodes in order to complete any of them, therefore, we go to (3).
3. While loop:

There are still arcs not yet sequenced. Following the current $VSEQ^1$ subsequence, open nodes 9, 10 and 14 lead to three additional open nodes in order to complete any of them. Let us choose node 10, arc (10,11), obtaining $VSEQ^1 = FPW_5 FPW_4 (8, 10) (8, 9) FPW_3 PW_2[(v_2, 14)] (10, 11)$. Following the subsequence $VSEQ^1$ obtained so far, no more arcs can be inserted that are incident solely to open nodes. Return to the beginning of while.

There are still arcs not yet sequenced. Following the current $VSEQ^1$ subsequence, open node 10 leads to two additional open nodes in order to complete it (node 11 is not considered since it is in T). We insert in $VSEQ^1$, say arc (10,6), obtaining $VSEQ^1 = FPW_5 FPW_4 (8, 10) (8, 9) FPW_3 PW_2[(v_2, 14)] (10, 11) (10, 6)$. Following the subsequence $VSEQ^1$ obtained so far, arc (6,9) can be inserted resulting in $VSEQ^1 = FPW_5 FPW_4 (8, 10) (8, 9) FPW_3 PW_2[(v_2, 14)] (10, 11) (10, 6) (6, 9)$. Return to the beginning of while.

There are still arcs not yet sequenced. Following the current $VSEQ^1$ subsequence, open node 10 leads to one additional open node in order to complete it (node 11 is not considered since it is in T^1). We insert in $VSEQ^1$, arc (10,16), obtaining $VSEQ^1 = FPW_5 FPW_4 (8, 10) (8, 9) FPW_3 PW_2[(v_2, 14)] (10, 11) (10, 6) (6, 9) (10, 16)$. Following the subsequence $VSEQ^1$ obtained so far, arcs (16,9) and (16,14) can be inserted resulting in $VSEQ^1 = FPW_5 FPW_4 (8, 10) (8, 9) FPW_3 PW_2[(v_2, 14)] (10, 11) (10, 6) (6, 9) (10, 16) (16, 9) (16, 14)$. Return to the beginning of while.

There are still arcs not yet sequenced. Following the current $VSEQ^1$ subsequence, open node 9 leads to one additional open node in order to complete it (node 11 is not considered since it is in T^1). We insert in $VSEQ^1$, arc (9,12), obtaining $VSEQ^1 = FPW_5 FPW_4 (8, 10) (8, 9) FPW_3 PW_2[(v_2, 14)] (10, 11) (10, 6) (6, 9) (10, 16) (16, 9) (16, 14) (9, 12)$. Following the subsequence $VSEQ^1$ obtained so far, arcs (11,12) and (12,14) can be inserted resulting in $VSEQ^1 = FPW_5 FPW_4 (8, 10) (8, 9) FPW_3 PW_2[(v_2, 14)] (10, 11) (10, 6) (6, 9) (10, 16) (16, 9) (16, 14) (9, 12) (11, 12) (12, 14)$. Return to the beginning of while.

There are still arcs not yet sequenced. Following the current $VSEQ^1$ subsequence, open node 14 leads to one additional open node in order to complete it (node 11 is not considered since

it is in T^1). We insert in $VSEQ^1$, arc (14,15), obtaining $VSEQ^1 = FPW_5 FPW_4 (8,10) (8,9) FPW_3 PW_2[(v_2,14)] (10,11) (10,6) (6,9) (10,16) (16,9) (16,14) (9,12) (11,12) (12,14) (14,15)$. Following the subsequence $VSEQ^1$ obtained so far, arcs (16,15) and (6,15) can be inserted resulting in $VSEQ^1 = FPW_5 FPW_4 (8,10) (8,9) FPW_3 PW_2[(v_2,14)] (10,11) (10,6) (6,9) (10,16) (16,9) (16,14) (9,12) (11,12) (12,14) (14,15) (16,15) (6,15)$. Return to the beginning of while.

There are still arcs not yet sequenced. Following the current $VSEQ^1$ subsequence, open node 16 (and also 6) leads to one additional open node in order to complete it (nodes 11 and 15 are not considered since they are in T^1). We insert in $VSEQ^1$, arc (16,13), obtaining $VSEQ^1 = FPW_5 FPW_4 (8,10) (8,9) FPW_3 PW_2[(v_2,14)] (10,11) (10,6) (6,9) (10,16) (16,9) (16,14) (9,12) (11,12) (12,14) (14,15) (16,15) (6,15) (16,13)$. Following the subsequence $VSEQ^1$ obtained so far, arc (13,12) can be inserted resulting in $VSEQ^1 = FPW_5 FPW_4 (8,10) (8,9) FPW_3 PW_2[(v_2,14)] (10,11) (10,6) (6,9) (10,16) (16,9) (16,14) (9,12) (11,12) (12,14) (14,15) (16,15) (6,15) (16,13) (13,12)$. Return to the beginning of while.

Following the same reasoning we may end up with: $VSEQ^1 = FPW_5 FPW_4 (8,10) (8,9) FPW_3 PW_2[(v_2,14)] (10,11) (10,6) (6,9) (10,16) (16,9) (16,14) (9,12) (11,12) (12,14) (14,15) (16,15) (6,15) (16,13) (13,12) (12,7) (7,6)$.

We amend together $VSEQ^1$ followed by $revVSEQO^2$, obtaining:

$FPW_5 FPW_4 (8,10) (8,9) FPW_3 PW_2[(v_2,14)] (10,11) (10,6) (6,9) (10,16) (16,9) (16,14) (9,12) (11,12) (12,14) (14,15) (16,15) (6,15) (16,13) (13,12) (12,7) (7,6) revFPW_1 revFPW_6$.

The resulting maximum number of open stacks using this sequence is 6.

Option B

Following this option we reach the sequence:

$FPW_6 FPW_1 (10,11) (11,12) (15,14) (12,14) (15,16) (16,14) (16,10) (15,6) (6,10) (16,13) (13,12) (16,9) (9,6) (9,12) (6,7) (7,12) (9,8) (8,10) revPW_2[(v_2,14)] revFPW_3 revFPW_4 revFPW_5$.

The resulting maximum number of open stacks using this sequence is also 6.

We, hence, may choose any one of these two solutions.

Observe that we cannot immediately guarantee that this solution is optimal for this example since the lower bound we obtained previously was 5. It is possible to show that when we have three (or more) subtrees in a graph attached to a node (or different nodes), each one of them with the same fpw-degree, say k , then the best we can do is to obtain a solution equal to $k+1$. This could be an additional way of determining a lower bound for the problem but this idea is not pursued further in this work since we will hardly find practical situations where this lower bound will prove useful. For this particular instance, however, since we have three subtrees with fpw-degree equal to 5 attached to three different nodes, this bound shows that an optimal solution value cannot be better than 6. Hence, the solution provided by the ArcContraction heuristic is optimal for the instance analysed.

The computational complexity of the ArcContraction heuristic is bounded by $O(n^3)$, where n is the total number of nodes in the problem. Determining which arcs to contract and doing the arc contractions (Step 1) are the most costly operations performed by the heuristic.

We present next some computational tests to analyse the performance of the proposed new lower bound and the Arc-contraction heuristic.

4. Computational experiments

To compare the lower bounds proposed, we performed some tests on some instances generated by hand and on randomly generated instances of MOSP. The random instances were generated with the following characteristics:

N = total number of patterns;

M = total number of different panel types;

C = maximum number of different panel types in each pattern.

For each pattern, we considered that the number of different panel types is uniform between $[1, C]$.

The Arc-contraction heuristic was implemented in C++ and was compared with an heuristic proposed by Yuen (1995). This last heuristic was also implemented in C++.

Yuen (1995) proposed several simple heuristics for the pattern sequencing problem. We implemented his third heuristic which he claimed produced the best results in his computational tests. Yuen's third heuristic is described next.

Define the following variables:

C_j : number of parts in common that pattern j has with stacks already open

N_j : number of new parts that pattern j is going to introduce if it is chosen

$M_j = C_j - N_j$

Heuristic 3

Choose to sequence next the non-sequenced pattern with largest M_j . If there is a tie, sequence next the pattern that introduces the smallest number of new parts (smallest N_j). If the tie persists, choose to sequence next the pattern with smallest index.

In Table 2 the results of the tests performed are presented. In the first column the data filename used is given. In the other columns the number of parts, number of patterns and C are indicated. In columns LB1, LB2, LB3, LB4 and LB5 the values of the lower bounds obtained are indicated. The results obtained by the new heuristic and Yuen's third heuristic are also indicated.

{Table 2}

From Table 2, we can see that the quality of lower bound LB^5 , as expected, is better than the previous bounds. The difference becomes more apparent as we increase the size of the problems. We can also see that the new proposed algorithm performed better than Yuen's third heuristic. Among all instances, only in one occasion Yuen's heuristic provided a better solution (Filename Caso604). However, the computational time of the new heuristic is much higher than Yuen's heuristic, but still within acceptable limits. The computational times listed in the Table 2 were obtained on a Pentium II 400 MHz, 128 MB RAM. Yuen's heuristic takes 2 to 3 ms for 30 patterns x 40 parts problems on a Pentium 166 MHz, 32 MB RAM. From the results, the running times seems to increase cubically with the number of nodes in the graph.

In the last column of Table 2, we provide an upper bound on the relative error of the new algorithm, calculated using the difference between the solution given by the new heuristic and LB^5 , divided by LB^5 . The actual error of the solution provided by the new heuristic is much smaller than the ones indicated. We tried to compute the exact solutions for all the instances, using a branch and bound method developed by the authors. The new heuristic did not provide the optimal solution in 11 cases and they all occurred for problems larger than 40 nodes. The maximum difference observed was 2 units in 5 of the cases (because of computational time limits set, for some of the larger problems, the branch and bound method was interrupted before assuring that the optimal solution value was reached). We

observe that although the quality of bound LB^5 is superior to the previous proposed ones, it is still poor, particularly for larger sized problems.

If the number of parts in the problem is maintained fixed and we vary the number of patterns, only the number of edges in the graph is modified. When the number of patterns increases, it is likely that the number of edges also increases in the graph. In Table 3 we present the results of the tests performed where we varied the number of edges in the graph. In the first column the data filename used is given. The filename has the form Gxxxxyzz, where xxx denotes the number of nodes, yy denotes the percentage of edges randomly deleted from the complete graph, and zz denotes the instance number. We can see that the lower bound LB^5 , as expected, is higher for graphs with a larger number of arcs. The number of edges in the graph seems to have little effect on the computational times of the heuristic. The computational times listed in the Table 3 were also obtained on a Pentium II 400 MHz, 128 MB RAM.

{Table 3}

5. SUMMARY AND CONCLUSIONS

Different possible ways of calculating lower bounds for the minimization of open stacks problem are discussed. A new procedure for determining a lower bound for MOSP was introduced. It uses the graph representation of the problem followed by an arc contraction procedure that reduces the graph in an attempt to make the analysis easier. The new procedure, as expected, found stronger bounds on all instances tested.

A polynomial exact algorithm is suggested for a particular pattern sequencing problem which is a complete graph with subtrees attached to its nodes. Its basic idea is used in the new proposed heuristic for solving the general pattern sequencing problem. This new polynomial heuristic uses the arc contraction scheme to generate an initial (incomplete) solution to MOSP. The heuristic proposed is quite simple and reasonably fast. Hence, if desired, the search being done can be extended to analyse other pattern sequencing possibilities.

Taking into account the computational tests performed, we can say that the quality of the solutions obtained is better than previous suggested heuristics.

The procedures presented can be used to generate good lower and upper bounds for the MOSP. They might be of great help in branch and bound schemes. The arc contraction heuristic basic idea can also be extended to generate other search schemes, different than the one suggested in Yanasse (1997a). This is currently being under investigation by these authors.

When focusing on the arc contraction scheme, the natural related issue that comes into mind is some sort of problem reduction. If two adjacent nodes i and j are such that the set of nodes adjacent to i (except for j) is a subset of the set of nodes adjacent to j (except for i), then it is always feasible to complete node i before completing node j , without increasing the number of open nodes. Therefore, a natural choice to perform would be to contract arc (i,j) . This choice, however, is not good for the purpose of finding stronger lower bounds based on *Proposition 3*. On the other hand, the contraction of such arcs might indicate a precedence order in which the nodes are to be finished. Consider, for instance, the graph presented in Figure 13. Nodes 2 and 3 are adjacent exactly to the same nodes, therefore, they can be finished one just after the other in any order; node 1 is adjacent to the same nodes as node 4, therefore, node 1 should be finished before node 4; nodes 2 and 3 are incident to the same nodes as node 4, therefore, nodes 2 and 3 should be finished before node 4. We believe that this information is quite relevant and may save computational efforts and time in branch-and-bound schemes. This is currently under investigation.

We finally observe that the contraction scheme can be used to simplify the procedures generated previously to solve optimally problems with special graph topologies. The case of trees, 1-trees and complete graphs are some of these special cases.

Acknowledgements: This work was partially funded by CNPq, a Brazilian Government Agency for Scientific and Technological Development and by FAPESP, São Paulo State Foundation for Research Support. We would like to thank the referees for their comments and suggestions that improved the presentation of the paper.

References

- (1) Foulds, L.R. *Graph Theory Applications*, Springer-Verlag, New York, 1992.
- (2) Garey; M.R.; Graham, R.L.; Johnson, D.S.; Knuth, D.E. "Complexity results for bandwidth minimization", *SIAM J. Appl. Math.*, 34, 477-495 (1978).
- (3) Garey; M.R.; Johnson, D.S. *Computers and Intractability: A Guide to the Theory of NP-Completeness*, Freeman, San Francisco, 1979.
- (4) Lins, S. "Traversing trees and scheduling tasks for duplex corrugator machines", *Pesquisa Operacional*, 9, 40-54 (1989).
- (5) Linhares, A. "Minimização de pilhas em aberto: alguns resultados de complexidade". Monografia de Exame de Qualificação de Doutorado, Curso CAP, INPE, maio de 1998.
- (6) Madsen, O.B.G. "An application of travelling-salesman routines to solve pattern-allocation problems in the glass industry", *Journal of the Operational Research Society*, 39, 249-256 (1988).
- (7) Tang, C.S.; Denardo, E.V. "Models arising from a flexible manufacturing machine, Part I: minimization of the number of tool switches", *Operations Research*, 36, 767-777 (1988).
- (8) Yanasse, H.H. "On a pattern sequencing problem in the wood industry". Presented at the TIMS XXXII, Anchorage, Alaska, June 1994.
- (9) Yanasse, H.H. "On a pattern sequencing problem to minimize the maximum number of open stacks", *European Journal of Operational Research*, 100:454-563, 1997a.
- (10) Yanasse, H.H. *A transformation for solving a pattern sequencing problem in the wood cut industry*, INPE/LAC Technical Report No. 006, 15p., 1996. *Pesquisa Operacional* 17(1):57-60, 1997b.
- (11) Yanasse, H.H. *Minimization of open orders - polynomial algorithms for some special cases*, INPE/LAC Technical Report No. 007, 1996. *Pesquisa Operacional* 16(1):1-26, 1996a.
- (12) Yanasse, H.H. *A heuristic procedure for solving a pattern sequencing problem in the wood cut industry*, Presented at the 2nd International Congress of Industrial Engineering / 16th National Congress of Production Engineering, São Carlos, São Paulo, Brazil, October 1996b.
- (13) Yanasse, H.H. An exact algorithm for the tree case of the minimization of open orders problem. Presented at the XXIX Simpósio Brasileiro de Pesquisa Operacional (SBPO), Salvador, BA, October 22 to 24, 1997c. INPE-LAC Technical Report 001/97.

- (14) Yuen, B.J. "Heuristics for sequencing cutting patterns", *European Journal of Operational Research*, 55, 183-190 (1991).
- (15) Yuen, B.J. "Improved heuristics for sequencing cutting patterns", *European Journal of Operational Research*, 87, 57-64 (1995).
- (16) Yuen, B.J.; Richardson, K.V. "Establishing the optimality of sequencing heuristics for cutting stock problems", *European Journal of Operational Research*, 84, 590-598 (1995).

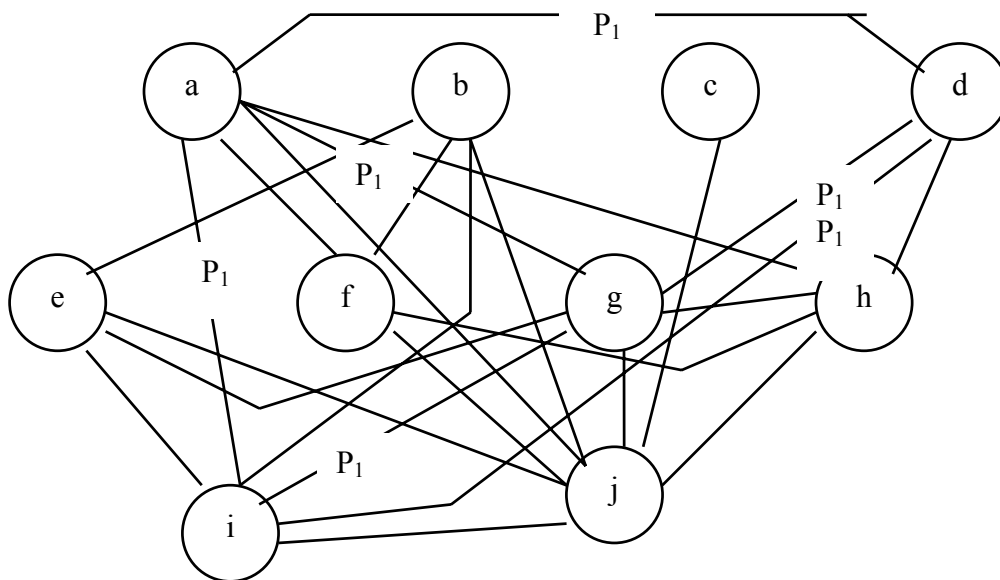


Figure 1
Graph $G(V,A)$, representation of problem data given in Table 1

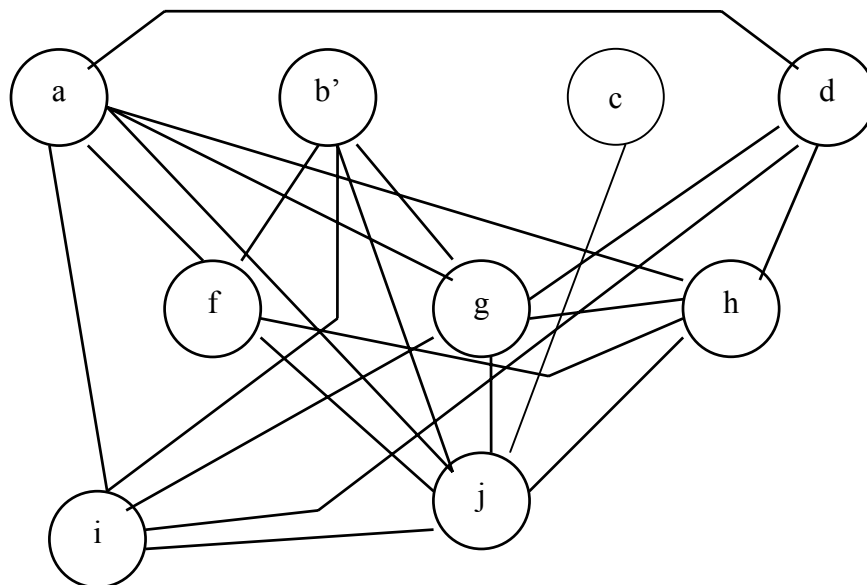


Figure 2
G/(b,e)

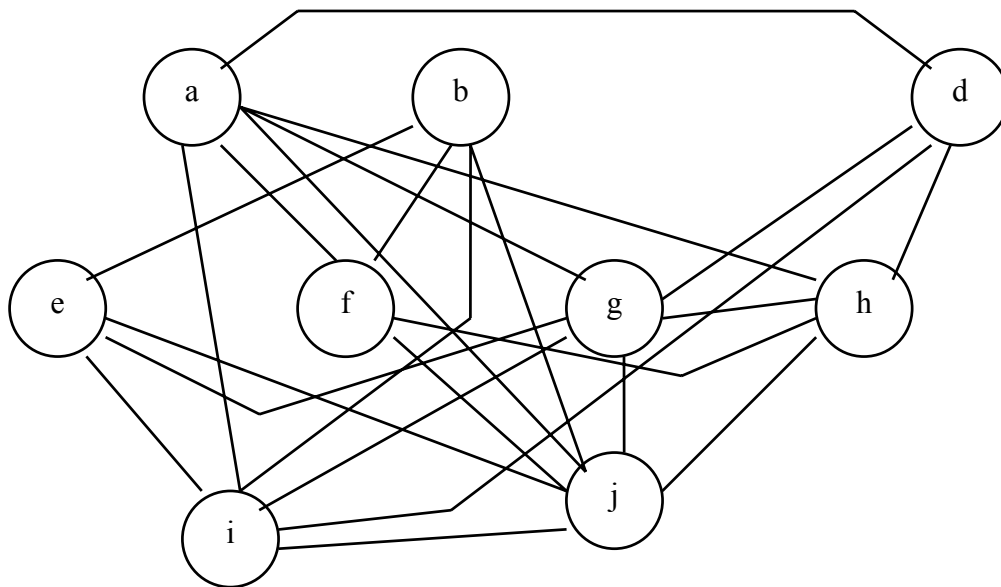


Figure 3
Graph obtained by deleting node c and all arcs incident to it

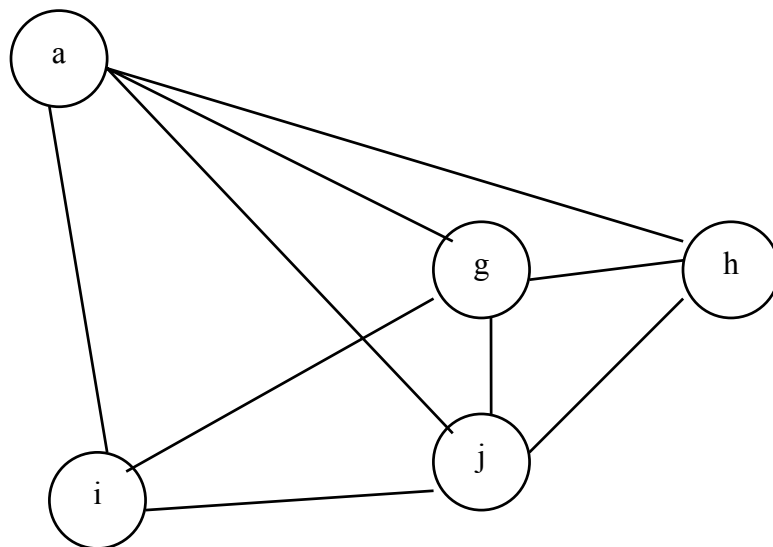


Figure 4

Graph obtained by deleting nodes b, d, e and f and all arcs incident to them

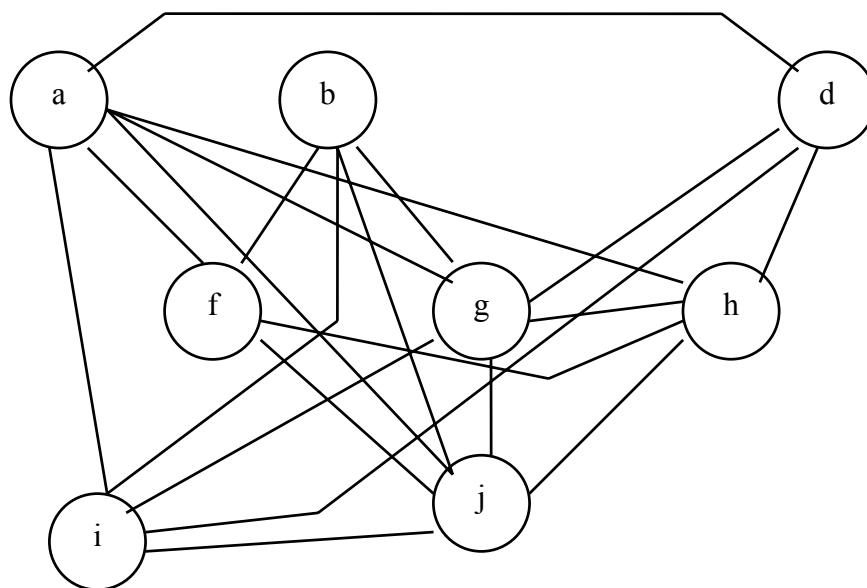


Figure 5
Graph obtained after contracting the arc that is incident to nodes b and e

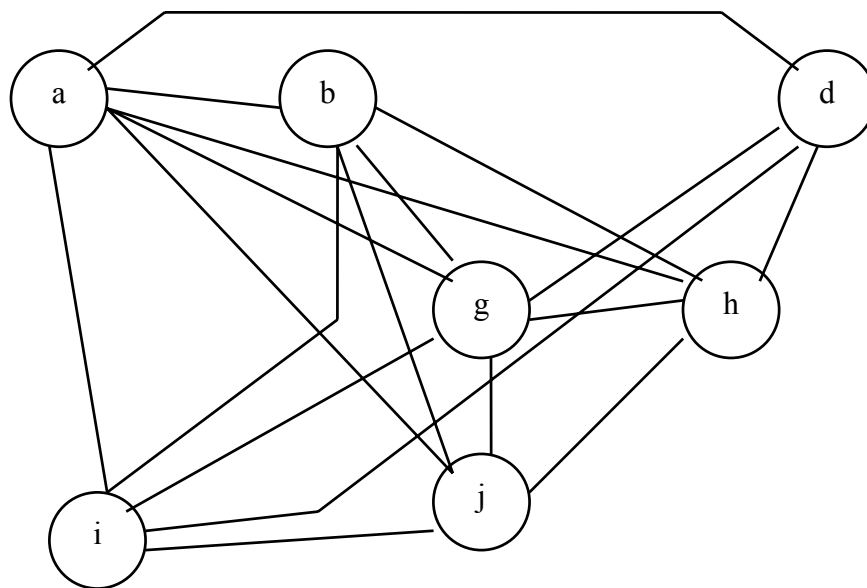


Figure 6
Graph obtained after contracting the arc that is incident to nodes b and f

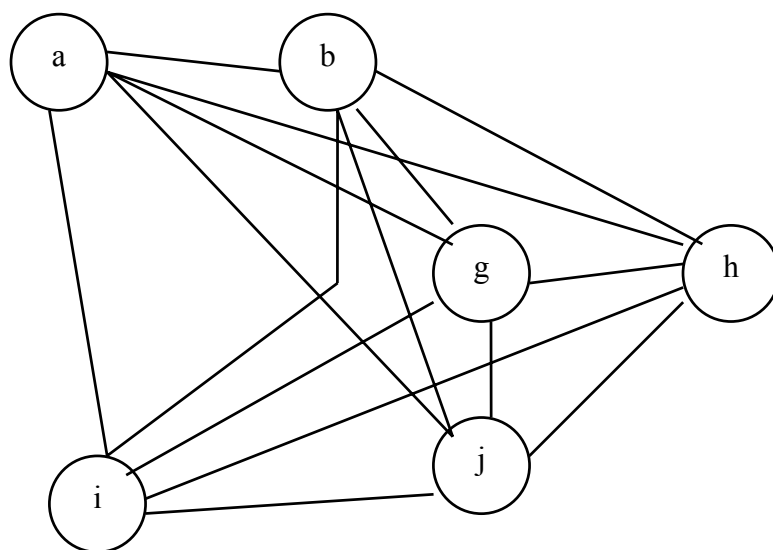


Figure 7
Graph obtained after contracting the arc that is incident to nodes d and h

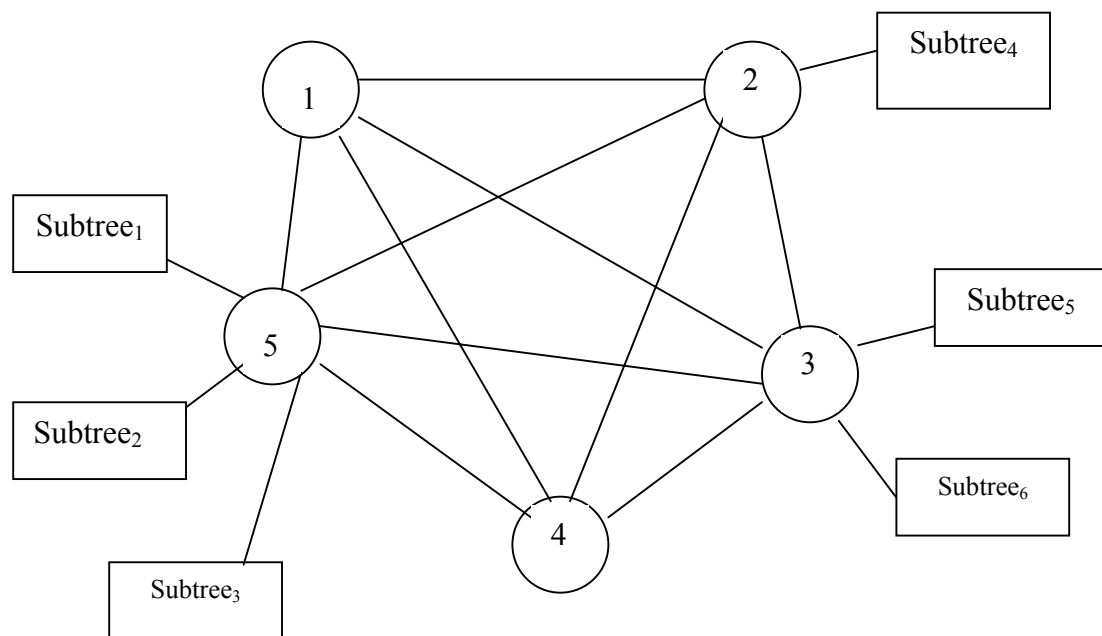


Figure 8
A complete subgraph having some trees attached to its nodes

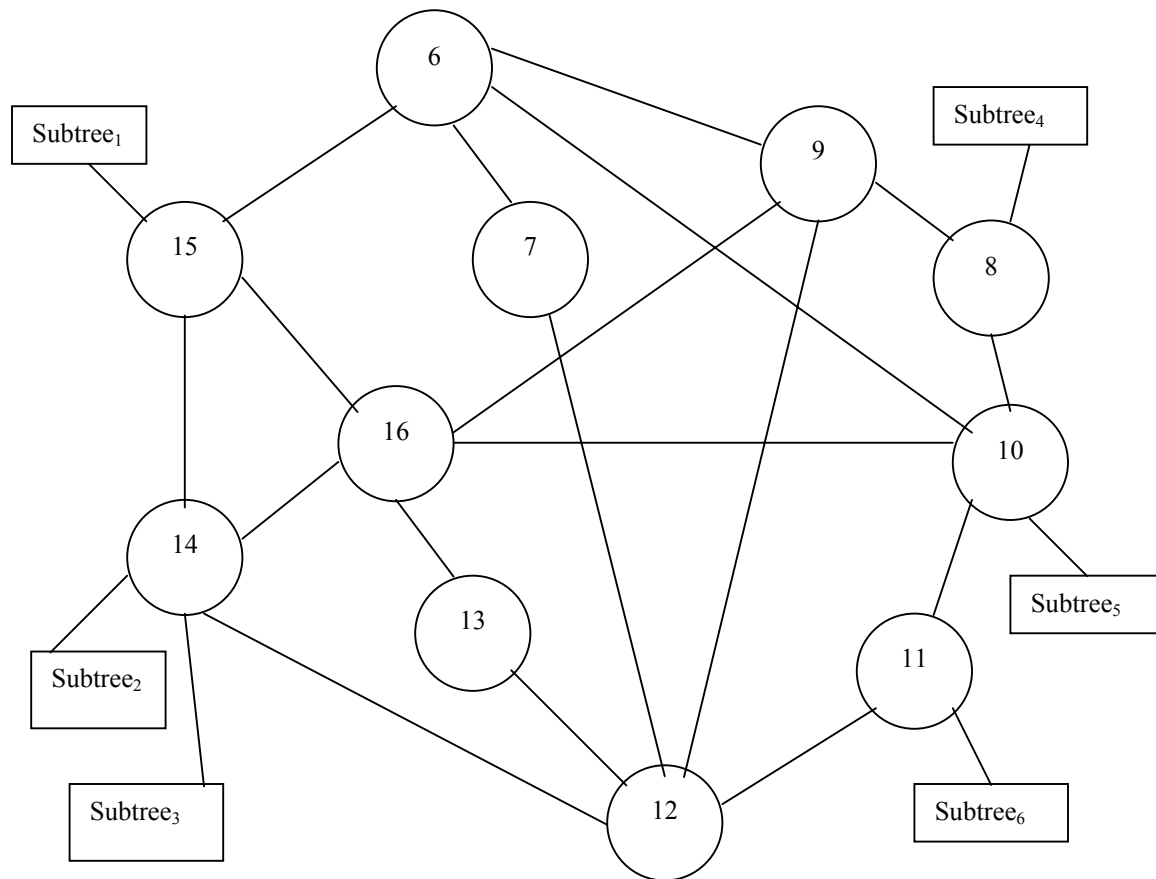


Figure 9
A graph corresponding to a general instance of MOSP

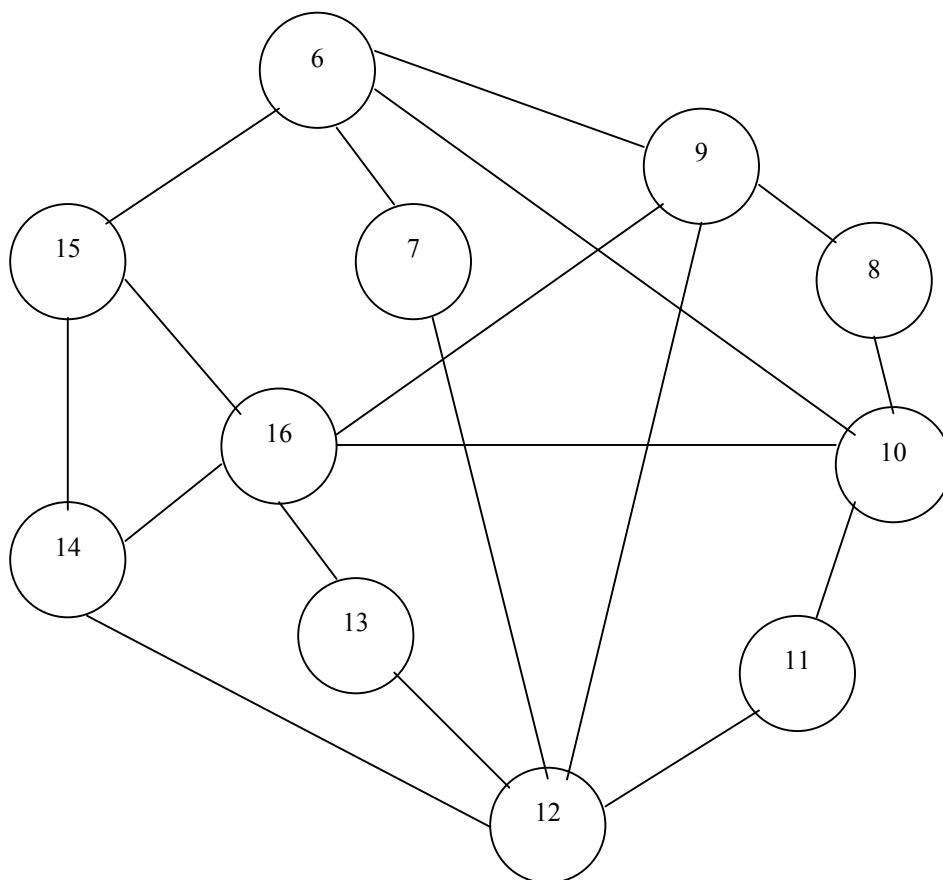


Figure 10
Subgraph¹ obtained after Step 0 of algorithm ArcContraction

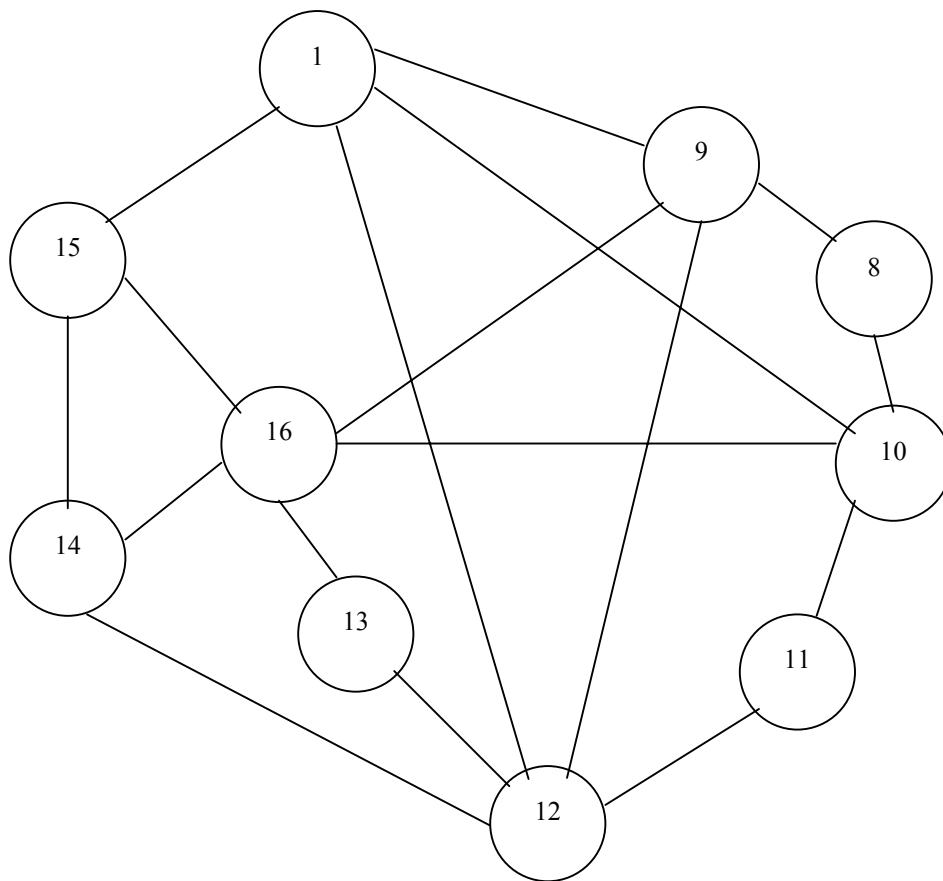


Figure 11
Subgraph² obtained after contraction arc (6,7)

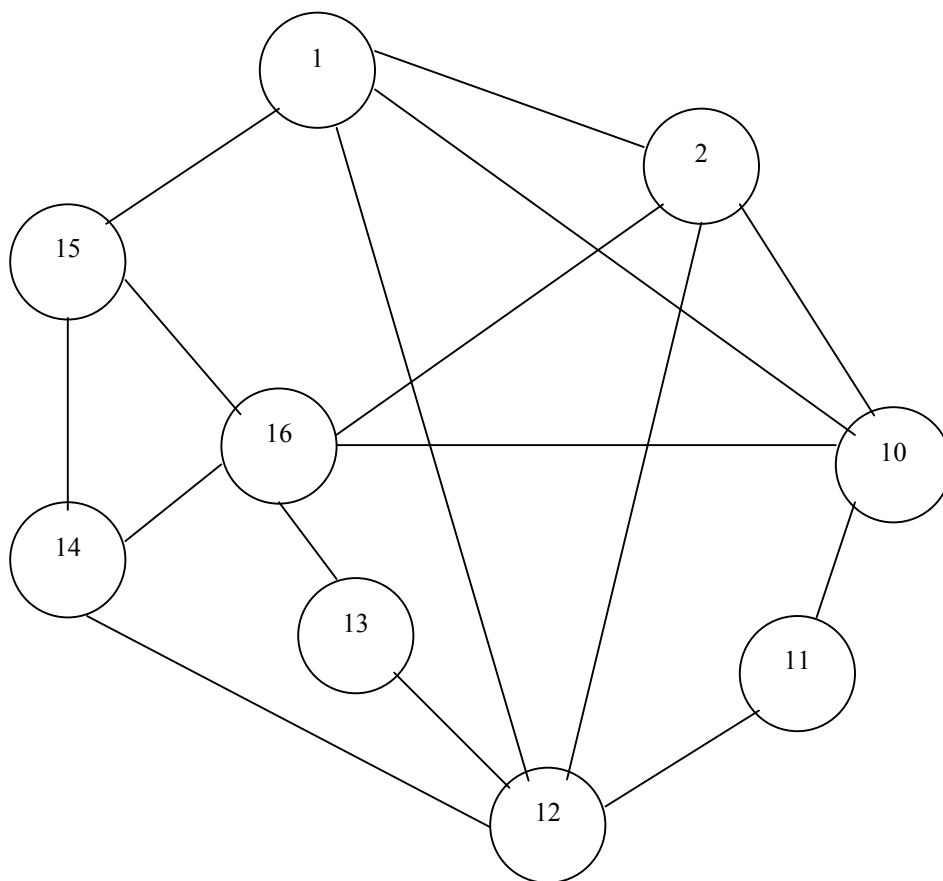


Figure 12
Subgraph³ obtained after contracting arc (8,9)

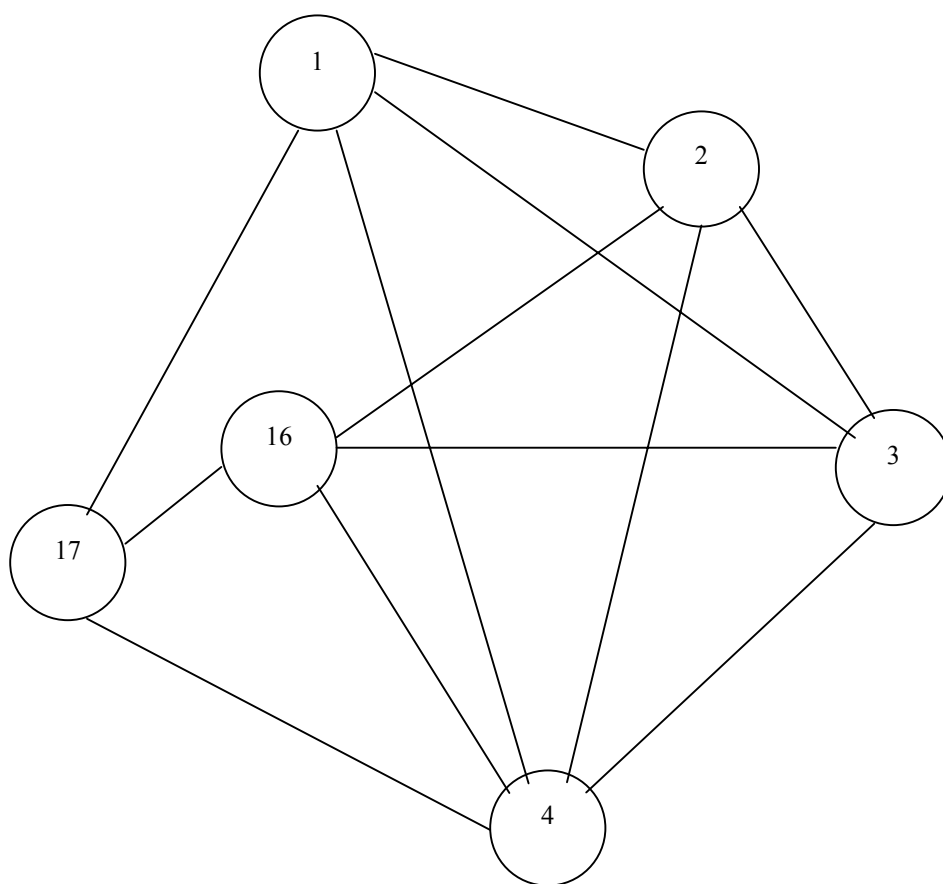


Figure 13
Subgraph⁶ obtained after contracting arcs (10,11), (12,13) and (14,15)

Table 1

Patterns	Order Parts
P ₁	a d g i
P ₂	d
P ₃	b e i j
P ₄	a f h j
P ₅	c j
P ₆	b f j
P ₇	d g h
P ₈	f
P ₉	a d
P ₁₀	e g i j

Table 2 – Results of the computational tests

Filename	Number of parts	Number of patterns	C	LB ¹	LB ²	LB ³	LB ⁴	LB ⁵	Contrac.	Time (sec.) Contrac.	Yuen's Heuristic	rel. error Contrac.
pattern0	06	04		04	04	04	04	04	04	0.01	05	0.00
pattern2	16	14		04	02	04	04	04	04	0.01	05	0.00
pattern4	20	18		04	02	04	04	04	04	0.01	05	0.00
pattern5	20	18		04	02	03	04	04	05	0.01	05	0.25
pattern6	18	16		04	02	04	04	04	04	0.01	05	0.00
pattern7	06	03		03	03	03	03	03	03	0.01	04	0.00
pattern8	16	22		02	02	03	04	04	06	0.01	07	0.50
pattern9	18	33		02	02	03	05	05	07	0.01	08	0.40
pattern10	18	35		02	02	05	05	05	06	0.01	09	0.20
pattern11	20	19		04	02	03	06	06	07	0.01	08	0.17
Caso101	10	10	04	04	04	04	06	06	06	0.01	07	0.00
Caso102	10	10	04	04	02	03	05	05	05	0.01	05	0.00
Caso103	10	10	04	04	03	03	05	05	06	0.01	06	0.20
Caso104	10	10	04	04	03	03	06	06	06	0.01	07	0.00
Caso201	20	15	08	08	05	05	11	11	12	0.05	14	0.09
Caso202	20	15	08	08	08	08	14	14	15	0.05	16	0.07
Caso203	20	15	08	08	03	03	13	13	14	0.06	16	0.08
Caso204	20	15	08	08	05	05	12	12	12	0.06	15	0.00
Caso301	30	25	10	10	04	04	19	19	21	0.17	24	0.11
Caso302	30	25	10	10	10	10	17	18	23	0.17	25	0.22
Caso303	30	25	10	10	05	05	18	19	22	0.11	27	0.14
Caso304	30	25	10	10	11	11	20	21	24	0.11	25	0.14
Caso401	40	30	16	15	23	23	30	32	34	0.33	38	0.06
Caso402	40	30	16	15	12	12	26	27	31	0.27	37	0.15
Caso403	40	30	16	15	22	22	27	29	34	0.38	38	0.17
Caso404	40	30	16	15	17	17	30	32	35	0.39	38	0.09

Filename	Number of parts	Number of patterns	C	LB ¹	LB ²	LB ³	LB ⁴	LB ⁵	Contrac.	Time (sec.) Contrac.	Yuen's Heuristic	rel. error Contrac.
Caso501	10	10	02	02	02	03	03	03	03	0.01	04	0.00
Caso502	10	10	02	02	02	03	00	03	03	0.01	03	0.00
Caso503	10	10	02	02	02	03	03	03	03	0.01	04	0.00
Caso504	10	10	02	07	02	04	07	07	07	0.01	08	0.00
Caso601	30	30	04	05	02	03	07	07	10	0.06	12	0.43
Caso602	30	30	04	04	03	03	06	07	12	0.01	13	0.71
Caso603	30	30	04	04	02	03	07	08	11	0.01	13	0.37
Caso604	30	30	04	05	02	03	06	07	12	0.05	11	0.71
Caso701	40	40	08	08	06	06	17	18	26	0.16	29	0.44
Caso702	40	40	08	08	03	03	14	15	22	0.17	24	0.47
Caso703	40	40	08	08	07	07	18	19	27	0.28	31	0.42
Caso704	40	40	08	08	04	04	16	18	26	0.22	33	0.44
Caso801	50	50	12	12	13	13	25	27	38	0.44	44	0.41
Caso802	50	50	12	12	18	18	30	32	42	0.66	45	0.31
Caso803	50	50	12	12	16	16	25	28	39	0.50	47	0.39
Caso804	50	50	12	12	15	15	28	31	40	0.61	47	0.29
Caso901	55	55	12	12	15	15	29	32	43	0.77	51	0.34
Caso902	55	55	12	12	05	05	28	30	41	0.66	50	0.37
Caso903	55	55	12	12	10	10	29	31	44	0.60	51	0.42
Caso904	55	55	12	12	14	14	29	32	43	0.65	49	0.34
Caso905	55	55	12	12	07	07	23	25	35	0.38	42	0.40
Caso906	55	55	12	12	23	23	30	33	45	0.71	49	0.36
Caso907	55	55	12	12	06	06	28	30	42	0.55	47	0.40
Caso908	55	55	12	12	06	06	27	29	41	0.77	47	0.41
CasoA01	60	60	12	12	13	13	27	30	42	2.31	51	0.40
CasoA02	60	60	12	12	14	14	28	31	44	0.71	51	0.42
CasoA03	60	60	12	12	15	15	31	35	47	0.88	54	0.34
CasoA04	60	60	12	12	20	20	32	35	48	1.37	56	0.37

Filename	Number of parts	Number of patterns	C	LB ¹	LB ²	LB ³	LB ⁴	LB ⁵	Contract.	Time (sec.) Contrac.	Yuen's Heuristic	rel. error Contract.
CasoB01	70	70	12	12	08	08	29	32	49	0.99	56	0.53
CasoB02	70	70	12	12	15	15	29	32	51	1.54	63	0.59
CasoB03	70	70	12	12	08	08	28	33	51	1.05	66	0.55
CasoB04	70	70	12	12	24	24	32	35	54	1.20	65	0.54
CasoC01	80	80	12	12	17	17	36	40	60	2.64	69	0.50
CasoC02	80	80	12	12	15	15	31	34	58	3.07	65	0.71
CasoC03	80	80	12	12	11	11	33	37	61	3.02	71	0.65
CasoC04	80	80	12	12	13	13	37	40	65	2.20	74	0.62
CasoD01	90	90	12	12	17	17	36	40	67	4.12	81	0.68
CasoD02	90	90	12	12	16	16	34	39	66	2.41	76	0.69
CasoD03	90	90	12	12	08	08	34	37	64	3.02	80	0.73
CasoD04	90	90	12	12	03	03	35	39	67	2.47	83	0.72
CasoE01	100	100	24	24	45	45	78	82	94	13.13	100	0.15
CasoE02	100	100	24	24	65	65	73	77	92	11.04	100	0.21
CasoE03	100	100	24	24	53	53	68	72	92	9.78	100	0.28
CasoE04	100	100	24	24	67	67	76	80	93	12.41	100	0.16
CasoF01	150	150	24	24	45	45	83	90	136	39.05	148	0.51
CasoF02	150	150	24	24	69	69	88	95	136	40.81	148	0.43
CasoF03	150	150	24	24	69	69	93	101	139	46.36	150	0.38
CasoF04	150	150	24	24	54	54	92	100	137	44.55	150	0.37

Table 3 – Results of the computational tests

Filename	LB ⁵	Contr.	Time	Filename	LB ⁵	Contr.	Time	Filename	LB ⁵	Contr.	Time	Filename	LB ⁵	Contr.	Time
G202001	14	17	0.01	G502001	36	45	0.55	G1002001	73	94	6.7	G1502001	110	143	37.5
G202002	14	16	0.01	G502002	35	45	0.55	G1002002	72	93	6.6	G1502002	111	143	36.0
G202003	14	16	0.01	G502003	36	45	0.54	G1002003	74	93	6.4	G1502003	111	143	38.1
G202004	14	16	0.01	G502004	35	44	0.55	G1002004	74	93	6.6	G1502004	111	143	35.0
G202005	14	17	0.01	G502005	36	45	0.55	G1002005	72	92	6.5	G1502005	111	143	36.4
G202006	14	16	0.05	G502006	36	44	0.60	G1002006	73	93	6.7	G1502006	111	143	36.3
G202007	14	17	0.06	G502007	36	45	0.49	G1002007	73	94	6.6	G1502007	109	143	37.4
G202008	14	16	0.05	G502008	36	45	0.55	G1002008	73	93	6.3	G1502008	110	143	38.1
G202009	14	16	0.06	G502009	36	45	0.54	G1002009	73	93	6.4	G1502009	112	142	36.0
G202010	14	17	0.05	G502010	36	44	0.55	G1002010	73	93	6.5	G1502010	110	143	35.2
G204001	11	13	0.01	G504001	27	42	0.55	G1004001	54	89	6.4	G1504001	81	137	35.9
G204002	12	15	0.01	G504002	27	41	0.49	G1004002	55	89	6.3	G1504002	82	138	34.9
G204003	11	13	0.01	G504003	27	41	0.55	G1004003	54	89	6.3	G1504003	82	137	31.6
G204004	11	14	0.01	G504004	27	42	0.55	G1004004	54	90	6.5	G1504004	82	138	31.0
G204005	11	14	0.01	G504005	27	41	0.49	G1004005	54	89	6.2	G1504005	81	138	35.9
G204006	11	15	0.01	G504006	27	42	0.50	G1004006	54	89	6.2	G1504006	82	138	35.2
G204007	11	14	0.05	G504007	27	42	0.55	G1004007	55	89	6.5	G1504007	81	138	31.9
G204008	11	14	0.06	G504008	27	42	0.50	G1004008	54	90	6.2	G1504008	81	138	31.9
G204009	11	15	0.05	G504009	27	40	0.55	G1004009	55	89	6.3	G1504009	81	138	32.1
G204010	11	14	0.01	G504010	28	41	0.55	G1004010	54	88	6.3	G1504010	81	138	31.
G206001	9	10	0.05	G506001	21	35	0.49	G1006001	39	82	6.1	G1506001	58	128	32.3
G206002	9	11	0.06	G506002	20	36	0.50	G1006002	38	81	6.3	G1506002	59	130	28.6
G206003	9	12	0.01	G506003	20	37	0.50	G1006003	39	82	6.3	G1506003	58	130	33.1
G206004	8	11	0.06	G506004	20	36	0.55	G1006004	39	83	6.1	G1506004	59	130	29.2
G206005	8	11	0.01	G506005	20	38	0.49	G1006005	40	83	6.0	G1506005	59	130	32.4
G206006	8	12	0.06	G506006	20	36	0.49	G1006006	39	83	6.0	G1506006	58	131	35.3
G206007	8	12	0.05	G506007	20	37	0.54	G1006007	39	82	7.2	G1506007	59	129	38.4
G206008	8	11	0.06	G506008	20	36	0.49	G1006008	39	81	6.1	G1506008	59	129	33.1
G206009	8	11	0.05	G506009	20	36	0.50	G1006009	39	83	6.1	G1506009	59	130	36.2
G206010	9	10	0.06	G506010	20	37	0.50	G1006010	39	81	6.1	G1506010	59	130	32.6

